



Studijski program: Informacione tehnologije

Informacioni sistem - Šahovska platforma

- Diplomski rad –

Mentor:

Dr Aleksandar Mišković

Student:

Dimitrije Mitrović 55/23

Beograd, 2026

Sadržaj

1. Uvod.....	1
2. Verbalni opis sistema.....	2
3. Specifikacija zahteva.....	3
3.1 Definicija korisničkih zahteva.....	3
3.2 Opis slučajeva korišćenja.....	6
3.3 Use case dijagrami.....	12
4. Analiza.....	14
4.1 Dijagrami sekvenci.....	14
4.2 Ugovori o sistemskim operacijama.....	19
4.3 Konceptualni model baze podataka.....	21
4.4 Relacioni model baze podataka.....	22
5. Projektovanje poslovne logike.....	25
5.1 Arhitektura i organizacija poslovne logike.....	25
5.2 Servisni sloj.....	26
5.3 Kolaboracioni dijagrami.....	28
6. Implementacija.....	36
6.1 Implementacija autentifikacije i autorizacije.....	36
6.2 Implementacija upravljanja turnirima.....	38
6.3 Implementacija upravljanja mečevima.....	40
6.4 Implementacija administratorskih funkcionalnosti.....	41
6.5 Implementacija statistike i tabele plasmana.....	43
7. Ručno testiranje.....	45
8. Zaključak.....	48
9. Literatura.....	50

1. Uvod

Razvoj informacionih sistema predstavlja važan segment savremenog softverskog inženjerstva, a veb aplikacije danas imaju značajnu ulogu u organizaciji i upravljanju različitim procesima. Zahvaljujući dostupnosti putem interneta i jednostavnom korisničkom interfejsu, ovakvi sistemi omogućavaju efikasnije upravljanje podacima, automatizaciju određenih poslovnih procesa i jednostavniju komunikaciju između korisnika.

Šah se tradicionalno igra na fizičkim lokacijama, posebno kada je reč o zvaničnim i takmičarskim turnirima. Međutim, razvoj digitalnih tehnologija omogućio je da se različiti aspekti organizacije šahovskih takmičenja prebace u elektronsko okruženje. Pored profesionalnih igrača, veliki broj korisnika šah posmatra kao oblik zabave, učenja i usavršavanja, zbog čega postoji potreba za jednostavnim sistemima koji mogu da olakšaju organizaciju i praćenje turnira.

Predmet ovog završnog rada jeste razvoj veb aplikacije namenjene organizaciji šahovskih turnira. Aplikacija omogućava upravljanje turnirima, registraciju i autentifikaciju korisnika, prijavljivanje igrača na turnire, formiranje parova za mečeve, unos rezultata partija, kao i prikaz plasmana i statistike učesnika. Fokus sistema nije implementacija same igre šaha, već organizacija turnira, vođenje evidencije o mečevima i rezultatima i upravljanje podacima u skladu sa definisanim poslovnim pravilima.

Prilikom razvoja aplikacije korišćene su tehnologije namenjene razvoju savremenih veb informacionih sistema. Aplikacija je implementirana korišćenjem programskog okvira ASP.NET Core MVC na .NET 9 platformi, dok je za rad sa bazom podataka korišćen Entity Framework Core 9 u okviru Code First pristupa. Za skladištenje podataka korišćen je Microsoft SQL Server, dok je autentifikacija i autorizacija korisnika realizovana primenom ASP.NET Core Identity mehanizma. Korisnički interfejs izrađen je uz pomoć Bootstrap okvira, uz dodatno korišćenje sopstvenih CSS stilova, čime je obezbeđen pregledan i prilagođen prikaz aplikacije na različitim veličinama ekrana.

Cilj ovog rada je da prikaže kompletan proces razvoja aplikacije, počevši od analize problema i definisanja zahteva, preko projektovanja sistema i modelovanja baze podataka, pa sve do implementacije i testiranja funkcionalnosti. Dokumentacija je izrađena u skladu sa Larmanovom metodom razvoja softvera i obuhvata opis sistema, specifikaciju zahteva, analizu sistema, UML dijagrame, projektovanje poslovne logike, implementaciju i ručno testiranje razvijene aplikacije.

2. Verbalni opis sistema

Razvijeni informacioni sistem predstavlja veb aplikaciju namenjenu organizaciji i upravljanju šahovskim turnirima. Osnovna svrha sistema jeste da omogući efikasno upravljanje turnirima, evidenciju prijavljenih igrača, formiranje parova za mečeve, unos rezultata odigranih partija i pregled plasmana i statistike učesnika. Sistem ne implementira samu igru šaha niti logiku kretanja figura, već je fokusiran na organizaciju i upravljanje turnirskim takmičenjem.

Sistem je namenjen radu više tipova korisnika, pri čemu su definisane tri korisničke uloge: administrator, organizator turnira i igrač. Svaka od ovih uloga poseduje odgovarajuća prava pristupa i skup funkcionalnosti koje su prilagođene njenim odgovornostima.

Administrator ima najviši nivo privilegija u sistemu. Njegova uloga podrazumeva upravljanje korisnicima i turnirima, pregled registrovanih korisnika, promenu korisničkih uloga, kao i brisanje korisnika i turnira. Na ovaj način administrator upravlja ključnim podacima sistema i kontroliše korisničke naloge.

Organizator turnira zadužen je za kreiranje i upravljanje turnirima. Prilikom kreiranja turnira unose se osnovni podaci, kao što su naziv, opis, datum početka i završetka, tip turnira i vremenska kontrola. Nakon što se prijavi dovoljan broj igrača i ispune definisani uslovi za pokretanje, organizator može pokrenuti turnir. Tom prilikom sistem formira parove igrača i kreira odgovarajuće mečeve.

Igrač predstavlja registrovanog korisnika sistema koji može pregledati dostupne turnire, prijaviti se na željeni turnir i odjaviti se pre njegovog pokretanja. Takođe može pregledati turnire na kojima učestvuje, svoje mečeve i njihove rezultate. Nakon odigranog meča, učesnik može evidentirati rezultat ukoliko su ispunjeni odgovarajući uslovi sistema.

Za svaki meč evidentiraju se učesnici, status meča i konačan rezultat. Rezultat može biti pobeda belog igrača, pobeda crnog igrača ili remi. Na osnovu evidentiranih rezultata sistem izračunava podatke potrebne za prikaz plasmana, uključujući broj odigranih mečeva, pobeda, remija, poraza i osvojenih poena.

Posebna pažnja posvećena je kontroli pristupa sistemu. Svaki korisnik nakon uspešne autentifikacije dobija pristup funkcionalnostima koje odgovaraju njegovoj korisničkoj ulozi. Na taj način omogućeno je razdvajanje privilegija između administratora, organizatora i igrača i sprečeno izvršavanje operacija za koje korisnik nema odgovarajuća prava.

Sistem je projektovan tako da omogući jednostavno korišćenje, pregledan korisnički interfejs i mogućnost proširivanja dodatnim funkcionalnostima u budućnosti. Organizacija aplikacije zasnovana je na MVC arhitekturi i servisnom sloju, čime je ostvarena jasna podela odgovornosti između korisničkog interfejsa, obrade zahteva, poslovne logike i pristupa podacima.

3. Specifikacija zahteva

3.1 Definicija korisničkih zahteva

Sistem za organizaciju šahovskih turnira razvijen je sa ciljem da omogućí jednostavno upravljanje turnirima, korisnicima i mečevima, uz jasno definisane korisničke uloge i njihove odgovornosti. Korisnički zahtevi predstavljaju funkcionalnosti koje sistem mora da ispuni kako bi omogućio nesmetan rad svim učesnicima.

U sistemu postoje tri osnovne korisničke uloge: administrator, organizator turnira i igrač. Svaka od ovih uloga raspolaže različitim skupom funkcionalnosti koje su definisane u skladu sa poslovnim pravilima sistema.

Administrator ima mogućnost pregleda svih registrovanih korisnika, izmene njihovih korisničkih uloga, brisanja korisničkih naloga, kao i upravljanja kreiranim turnirima. Administrator takođe može pregledati i uklanjati turnire iz sistema. Na ovaj način omogućeno je centralizovano upravljanje korisničkim nalogima i podacima o turnirima.

Organizator turnira ima mogućnost kreiranja novih turnira unosom osnovnih podataka, kao što su naziv, opis, tip turnira, vremenska kontrola i period održavanja turnira. Takođe može pregledati turnire koje je kreirao, pokrenuti turnir kada su ispunjeni definisani uslovi i obrisati turnire kojima upravlja.

Registrovani igrač može da se prijavi u sistem, pregleda dostupne turnire, prijavi se na željeni turnir ili odustane od prijave pre njegovog pokretanja. Nakon pokretanja turnira igrač može pregledati svoje mečeve, otvoriti detalje pojedinačnog meča i evidentirati njegov rezultat ukoliko je učesnik tog meča. Na osnovu završenih mečeva sistem omogućava prikaz podataka o plasmanu učesnika.

Pored funkcionalnih zahteva, sistem mora zadovoljiti i osnovne nefunkcionalne zahteve. Pristup funkcionalnostima mora biti ograničen korisničkim ulogama, podaci moraju biti trajno sačuvani u bazi podataka, a korisnički interfejs treba da bude pregledan i prilagođen radu na različitim veličinama ekrana. Sistem takođe mora obezbediti validaciju korisničkog unosa i primenu definisanih poslovnih pravila kako bi se sprečio unos neispravnih ili nedozvoljenih podataka.

Tabela 1. Korisnički zahtevi

Oznaka	Zahtev	Korisnik
Z-01	Registracija korisnika	Neregistrovani korisnik
Z-02	Prijava na sistem	Registrovani korisnik
Z-03	Kreiranje turnira	Organizator
Z-04	Prijava na turnir	Igrač
Z-05	Napuštanje turnira	Igrač
Z-06	Pokretanje turnira	Organizator
Z-07	Pregled mečeva	Igrač
Z-08	Evidentiranje rezultata	Igrač
Z-09	Pregled plasmana	Korisnik
Z-10	Upravljanje korisnicima	Administrator
Z-11	Upravljanje turnirima	Administrator

Tabela 2. Korisničke uloge I funkcionalnosti

Funkcionalnost	Administrator	Organizator	Igrač
Pregled sistema	✓	✓	✓
Upravljanje korisnicima	✓	–	–
Kreiranje turnira	–	✓	–
Brisanje turnira	✓	✓	–
Pokretanje turnira	–	✓	–
Prijava na turnir	–	--	✓
Napuštanje turnira	–	--	✓

Funkcionalnost	Administrator	Organizator	Igrač
Pregled mečeva	–	–	✓
Unos rezultata	–	–	✓
Pregled plasmana	--	✓	✓

3.2 Opis slučaja korišćenja

UC-1 Registracija korisnika

Akter: Neregistrovani korisnik

Opis: Korisnik kreira novi nalog unosom osnovnih podataka.

Preduslov: Korisnik nije prijavljen u sistem.

Osnovni tok:

1. Korisnik otvara stranicu za registraciju.
2. Unosi korisničko ime, ime i prezime, datum rođenja, e-mail adresu i lozinku.
3. Sistem proverava ispravnost unetih podataka.
4. Sistem kreira novi korisnički nalog i dodeljuje mu ulogu Player.
5. Nakon uspešne registracije sistem automatski prijavljuje korisnika i preusmerava ga na odgovarajuću stranicu.

Alternativni tok:

- Ukoliko su podaci neispravni ili korisničko ime odnosno e-mail adresa već postoje, sistem prikazuje odgovarajuću poruku o grešci i korisnik mora ispraviti podatke.

Postuslov: Novi korisnički nalog je uspešno kreiran i korisnik je prijavljen u sistem.

UC-2 Prijava korisnika

Akter: Registrovani korisnik

Opis: Korisnik se prijavljuje u sistem korišćenjem korisničkog imena i lozinke.

Preduslov: Korisnik poseduje registrovan nalog.

Osnovni tok:

1. Korisnik otvara formu za prijavu.
2. Unosi korisničko ime i lozinku.
3. Sistem proverava validnost unetih podataka.
4. Nakon uspešne autentifikacije sistem utvrđuje korisničku ulogu.
5. Korisnik se preusmerava na početnu stranicu odgovarajuće korisničke uloge.

Alternativni tok:

- Ukoliko su podaci za prijavu pogrešni, sistem odbija prijavu i prikazuje poruku o grešci.

Postuslov: Korisnik je uspešno prijavljen u sistem.

UC-3 Kreiranje turnira

Akter: Organizator turnira

Opis: Organizator kreira novi turnir definisanjem njegovih osnovnih karakteristika.

Preduslov: Organizator je prijavljen u sistem.

Osnovni tok:

1. Organizator bira opciju za kreiranje turnira.
2. Unosi naziv i opis turnira.
3. Bira tip turnira i vremensku kontrolu.
4. Unosi datum početka i datum završetka turnira.
5. Sistem proverava validnost unetih podataka.
6. Sistem proverava da li je datum završetka nakon datuma početka.
7. Novi turnir se čuva u bazi podataka.
8. Organizator se preusmerava na pregled svojih turnira.

Alternativni tok:

- Ukoliko podaci nisu validni ili datum završetka nije nakon datuma početka, sistem prikazuje odgovarajuću poruku o grešci i ne kreira turnir.

Postuslov: Turnir je uspešno kreiran i sačuvan u bazi podataka.

UC-4 Prijava igrača na turnir

Akter: Igrač

Opis: Igrač se prijavljuje na željeni turnir.

Preduslov: Igrač je prijavljen u sistem, a turnir nije pokrenut.

Osnovni tok:

1. Igrač pregledava dostupne turnire.
2. Bira turnir na kojem želi da učestvuje.

3. Klikom na dugme za prijavu pokreće proces prijavljivanja.
4. Sistem proverava da li je igrač već prijavljen na izabrani turnir.
5. Sistem evidentira prijavu igrača.
6. Igrač se prikazuje kao učesnik izabranog turnira.

Alternativni tok:

- Ukoliko je igrač već prijavljen na turnir, sistem ne dozvoljava ponovnu prijavu.
- Ukoliko je turnir već pokrenut, sistem ne dozvoljava prijavljivanje.

Postuslov: Igrač je uspešno prijavljen na turnir.

UC-5 Napuštanje turnira

Akter: Igrač

Opis: Igrač se odjavljuje sa turnira na kojem je prethodno prijavljen.

Preduslov: Igrač je prijavljen na turnir, a turnir još nije pokrenut.

Osnovni tok:

1. Igrač otvara pregled svojih turnira.
2. Bira turnir sa kojeg želi da se odjavi.
3. Pokreće opciju za napuštanje turnira.
4. Sistem pronalazi njegovu prijavu i uklanja je.
5. Ažurira se lista turnira na kojima igrač učestvuje.

Alternativni tok:

- Ukoliko igrač nije prijavljen na izabrani turnir, sistem ne izvršava operaciju.
- Ukoliko je turnir već pokrenut, igrač se ne može odjaviti.

Postuslov: Igrač više nije prijavljen na izabrani turnir.

UC-6 Pokretanje turnira

Akter: Organizator turnira

Opis: Organizator pokreće turnir nakon ispunjavanja definisanih uslova, pri čemu sistem kreira početne mečeve.

Preduslov: Organizator je vlasnik turnira i turnir još nije pokrenut.

Osnovni tok:

1. Organizator bira opciju za pokretanje turnira.
2. Sistem proverava da li organizator ima pravo da pokrene izabrani turnir.
3. Sistem proverava broj prijavljenih igrača.
4. Sistem proverava da li je broj igrača najmanje dva i paran.
5. Sistem formira parove prijavljenih igrača.
6. Za svaki formirani par kreira se novi meč.
7. Turnir se označava kao pokrenut.

Alternativni tok:

- Ukoliko organizator nije kreirao izabrani turnir, operacija se odbija.
- Ukoliko je turnir već pokrenut, operacija se odbija.
- Ukoliko je prijavljeno manje od dva igrača, turnir se ne može pokrenuti.
- Ukoliko je broj prijavljenih igrača neparan, turnir se ne može pokrenuti.

Postuslov: Turnir je označen kao pokrenut i kreirani su mečevi za prijavljene igrače.

UC-7 Pregled mečeva

Akter: Igrač

Opis: Igrač pregleda mečeve koji su mu dodeljeni u okviru pokrenutog turnira.

Preduslov: Igrač učestvuje u pokrenutom turniru i za njega postoji meč.

Osnovni tok:

1. Igrač otvara stranicu „My Matches“.
2. Sistem pronalazi mečeve u kojima učestvuje prijavljeni korisnik.
3. Sistem prikazuje protivnika, boju figura, status i rezultat meča.
4. Igrač može otvoriti detalje izabranog meča.

Postuslov: Igrač ima pregled svojih dodeljenih mečeva i njihovog trenutnog statusa.

UC-8 Evidentiranje rezultata meča

Akter: Igrač

Opis: Igrač evidentira rezultat meča u kojem učestvuje.

Preduslov: Igrač učestvuje u konkretnom meču i meč nije prethodno završen.

Osnovni tok:

1. Igrač otvara detalje meča.
2. Bira jedan od ponuđenih ishoda: pobjeda belog igrača, pobjeda crnog igrača ili remi.
3. Sistem proverava da li je igrač učesnik meča.
4. Sistem proverava da li je izabrani rezultat dozvoljen.
5. Sistem čuva rezultat meča.
6. Sistem određuje pobjednika ukoliko je rezultat pobjeda jednog od igrača.
7. Sistem označava meč kao završen.
8. Podaci o završenom meču koriste se za formiranje plasmana.

Alternativni tok:

- Ukoliko igrač nije učesnik meča, sistem ne dozvoljava unos rezultata.
- Ukoliko je meč već završen, sistem ne dozvoljava promenu rezultata.
- Ukoliko rezultat nije jedan od dozvoljenih ishoda, sistem odbija operaciju.

Postuslov: Rezultat meča je sačuvan, a meč je označen kao završen.

UC-9 Pregled plasmana

Akter: Korisnik

Opis: Korisnik pregleda tabelu plasmana učesnika izabranog turnira.

Preduslov: Postoji turnir sa prijavljenim igračima.

Osnovni tok:

1. Korisnik otvara stranicu plasmana izabranog turnira.
2. Sistem pronalazi prijavljene igrače.
3. Sistem pronalazi završene mečeve tog turnira.
4. Za svakog igrača izračunava broj odigranih mečeva, pobjeda, remija, poraza i osvojenih poena.
5. Sistem sortira igrače prema broju osvojenih poena i broju pobjeda.
6. Korisniku se prikazuje tabela plasmana.

Postuslov: Prikazana je ažurirana tabela plasmana učesnika.

UC-10 Upravljanje korisnicima

Akter: Administrator

Opis: Administrator upravlja registrovanim korisnicima sistema.

Preduslov: Administrator je prijavljen u sistem.

Osnovni tok:

1. Administrator otvara listu korisnika.
2. Sistem prikazuje registrovane korisnike i njihove korisničke uloge.
3. Administrator bira željenu akciju.
4. Administrator može promeniti korisničku ulogu ili obrisati korisnički nalog.
5. Sistem proverava da li je izabrana operacija dozvoljena.
6. Sistem izvršava izabranu operaciju i prikazuje ažuriranu listu korisnika.

Alternativni tok:

- Administrator ne može obrisati sopstveni korisnički nalog.
- Ukoliko izabrani korisnik ne postoji, sistem ne izvršava operaciju.

Postuslov: Korisnički podaci su ažurirani u skladu sa izvršenom operacijom.

UC-11 Upravljanje turnirima od strane administratora

Akter: Administrator

Opis: Administrator pregleda i upravlja turnirima koji postoje u sistemu.

Preduslov: Administrator je prijavljen u sistem.

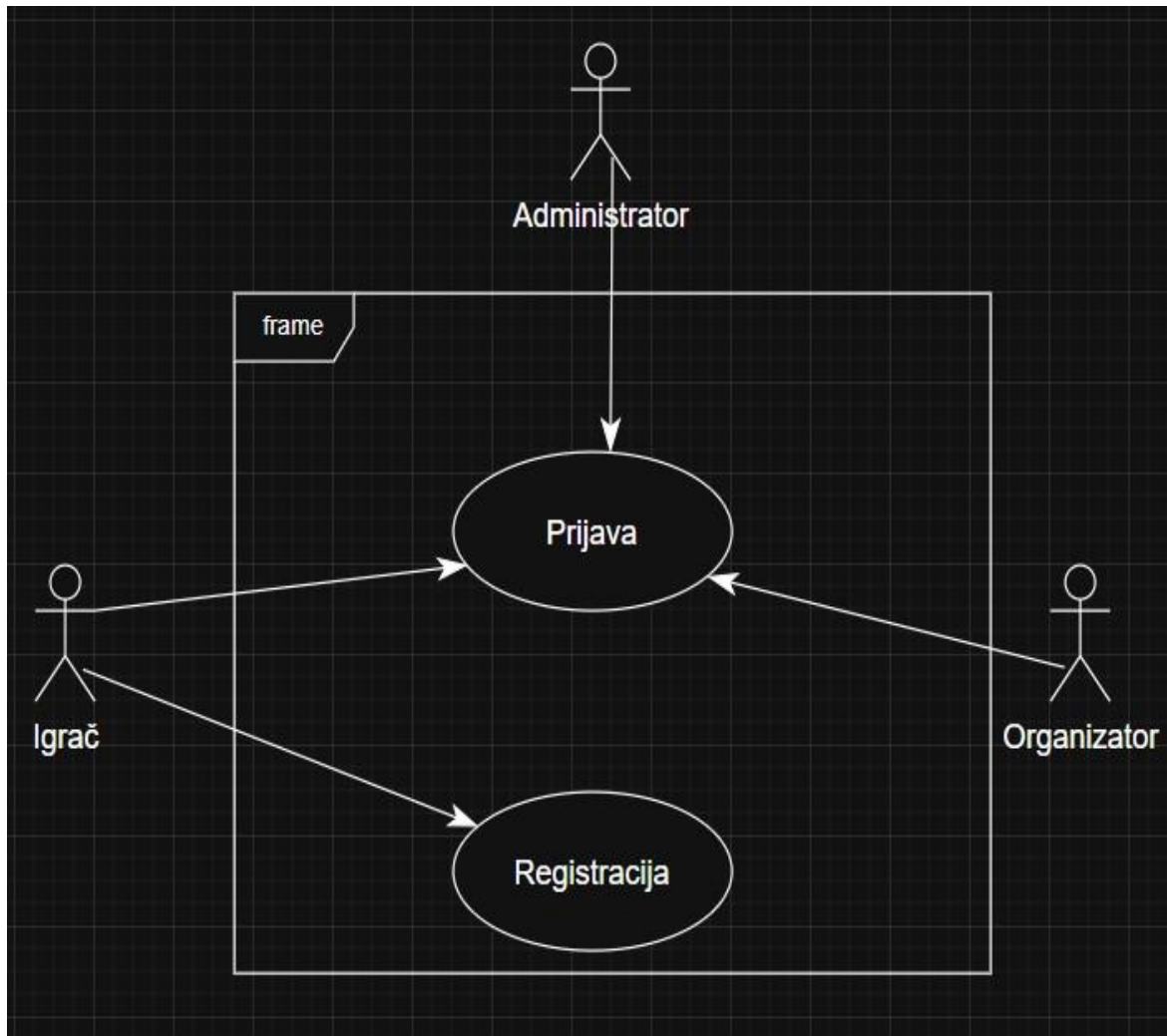
Osnovni tok:

1. Administrator otvara pregled turnira.
2. Sistem prikazuje turnire registrovane u sistemu.
3. Administrator bira turnir nad kojim želi da izvrši dozvoljenu akciju.
4. Sistem izvršava izabranu operaciju.
5. Sistem prikazuje ažurirani pregled turnira.

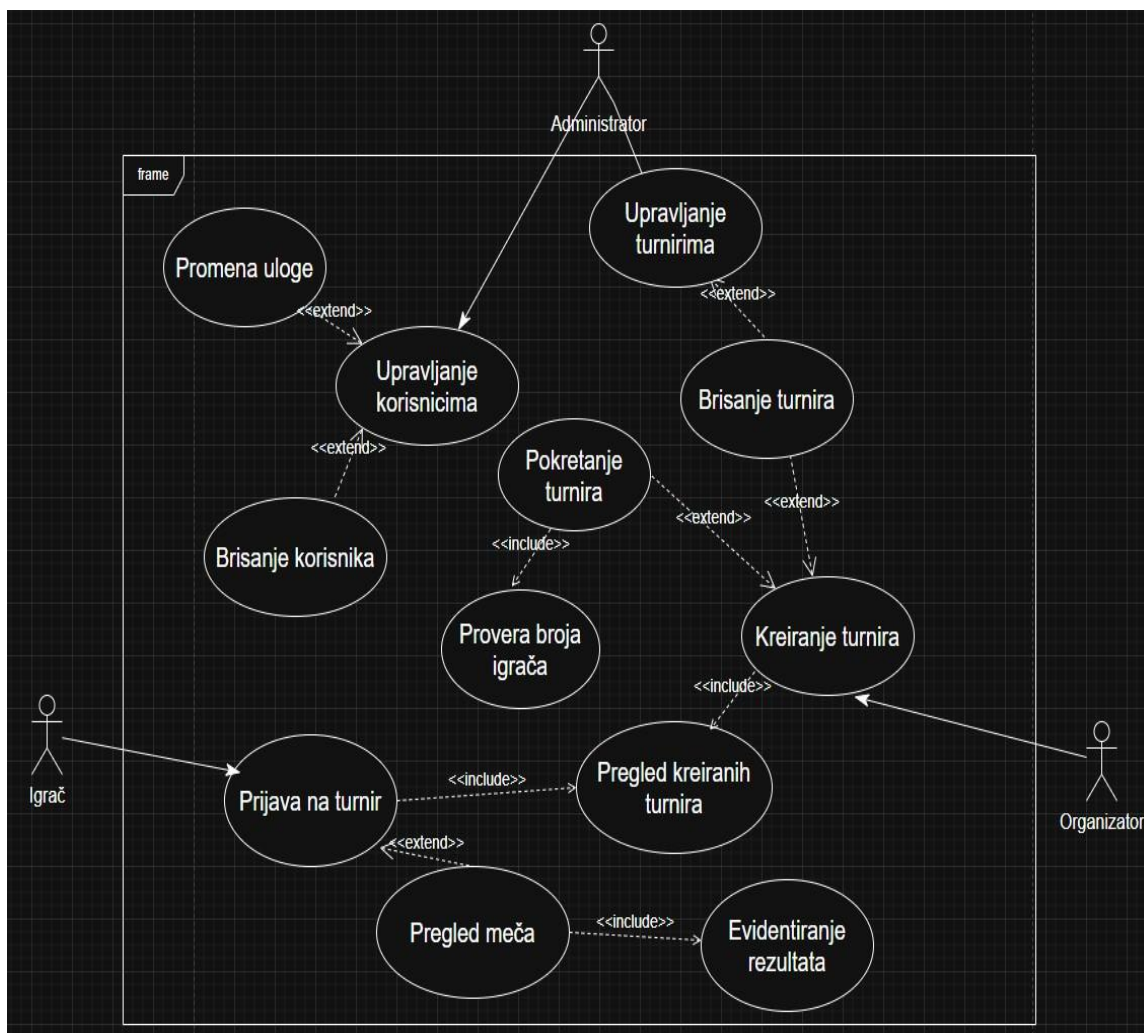
Postuslov: Podaci o turnirima su ažurirani u skladu sa izvršenom operacijom.

3.3 Use case dijagrami

U nastavku će biti prikazani use case dijagrami koji predstavljaju funkcionalnosti sistema iz perspektive različitih korisničkih uloga. Dijagrami obuhvataju osnovne funkcionalnosti dostupne neregistrovanim korisnicima, registrovanim igračima, organizatorima turnira i administratorima.



Use case dijagram pristupa platformi



Use case diagram funkcija uloga

4. Analiza

Analiza sistema predstavlja jednu od najvažnijih faza razvoja softvera, jer omogućava detaljno sagledavanje načina na koji sistem izvršava definisane funkcionalnosti. Na osnovu prethodno definisanih korisničkih zahteva i slučajeva korišćenja, u ovoj fazi modeluju se ključne sistemske operacije i međusobna komunikacija između korisnika i pojedinih komponenti sistema.

Cilj analize jeste da se definiše ponašanje sistema pre same implementacije, čime se obezbeđuje jasnije razumevanje poslovne logike i smanjuje mogućnost grešaka tokom razvoja aplikacije. U okviru ovog poglavlja biće prikazani dijagrami sekvenci za najvažnije slučajeve korišćenja, ugovori sistemskih operacija, kao i konceptualni i relacioni model baze podataka.

Sekvencni dijagrami prikazuju redosled razmene poruka između korisnika i sistema tokom izvršavanja određene funkcionalnosti, dok ugovori sistemskih operacija definišu ulazne parametre, preduslove i rezultate izvršavanja svake operacije. Pored toga, konceptualni model predstavlja osnovne entitete sistema i njihove međusobne veze, dok relacioni model prikazuje strukturu baze podataka koja je korišćena prilikom implementacije aplikacije.

4.1 Dijagrami sekvenci

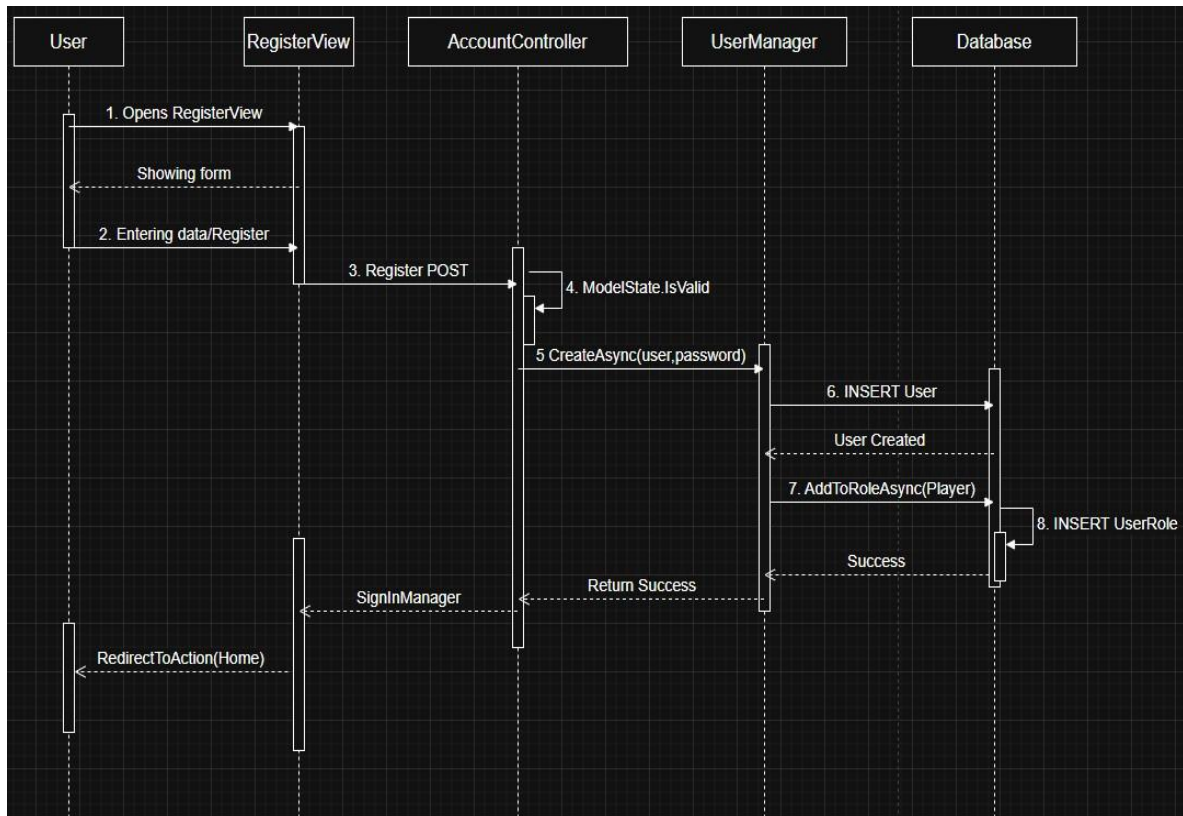
Dijagram sekvenci – Registracija korisnika

Sekvencni dijagram registracije korisnika prikazuje tok izvršavanja procesa kreiranja novog korisničkog naloga u sistemu. Proces započinje kada korisnik otvori stranicu za registraciju i unese tražene podatke, kao što su korisničko ime, ime i prezime, datum rođenja, e-mail adresa i lozinka. Nakon slanja forme, podaci se prosleđuju kontroleru koji vrši validaciju unetih vrednosti.

Ukoliko su svi podaci ispravni i ne postoji korisnik sa istim korisničkim imenom ili e-mail adresom, kontroler poziva ASP.NET Core Identity mehanizam za kreiranje novog korisničkog naloga. Nakon uspešnog upisa podataka u bazu, korisniku se dodeljuje podrazumevana uloga Player, čime stiže mogućnost korišćenja funkcionalnosti sistema namenjenih registrovanim igračima.

Po završetku procesa registracije sistem obaveštava korisnika o uspešno kreiranom nalogu i omogućava mu prijavu u sistem. Ukoliko validacija nije uspešna ili dođe do greške prilikom kreiranja naloga, korisniku se prikazuje odgovarajuća poruka o grešci, a postupak registracije se prekida dok se nepravilnosti ne otklone.

Prikazani sekvencni dijagram jasno ilustruje komunikaciju između korisnika, korisničkog interfejsa, kontrolera, Identity servisa i baze podataka tokom procesa registracije.



Dijagram sekvenci – Registracija korisnika

Sekvenčni dijagram – Kreiranje turnira

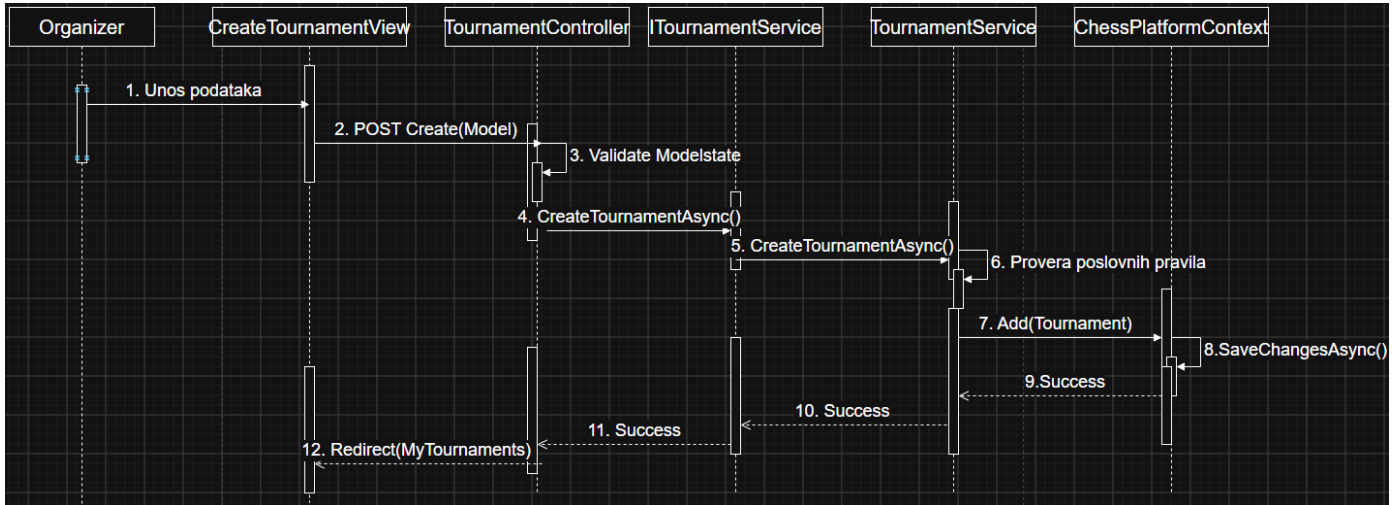
Sekvenčni dijagram kreiranja turnira prikazuje tok izvršavanja procesa kreiranja novog turnira od strane organizatora. Proces započinje kada organizator otvori stranicu za kreiranje turnira i unese podatke, kao što su naziv, opis, tip turnira, vremenska kontrola, datum početka i datum završetka turnira. Nakon slanja forme, podaci se prosleđuju kontroleru koji vrši osnovnu validaciju unetih vrednosti.

Ukoliko su podaci ispravni, kontroler prosleđuje zahtev servisnom sloju, odnosno interfejsu `ITournamentService`, koji poziva odgovarajuću implementaciju `TournamentService`. Servisni sloj izvršava poslovne provere, uključujući proveru ispravnosti datuma, a zatim kreira novi objekat turnira i prosleđuje ga kontekstu baze podataka `ChessPlatformContext` radi čuvanja.

Nakon uspešnog upisa podataka u bazu, sistem potvrđuje uspešno izvršenu operaciju i organizatora preusmerava na stranicu sa pregledom njegovih turnira, gde se novokreirani turnir prikazuje na listi.

U slučaju da validacija podataka ili poslovna pravila nisu zadovoljeni, sistem ne kreira novi turnir, već ponovo prikazuje formu za unos zajedno sa odgovarajućim porukama o greškama kako bi organizator mogao da ispravi nepravilno unete podatke.

Prikazani sekvencni dijagram ilustruje komunikaciju između organizatora, korisničkog interfejsa, kontrolera, servisnog sloja i konteksta baze podataka tokom procesa kreiranja novog turnira.



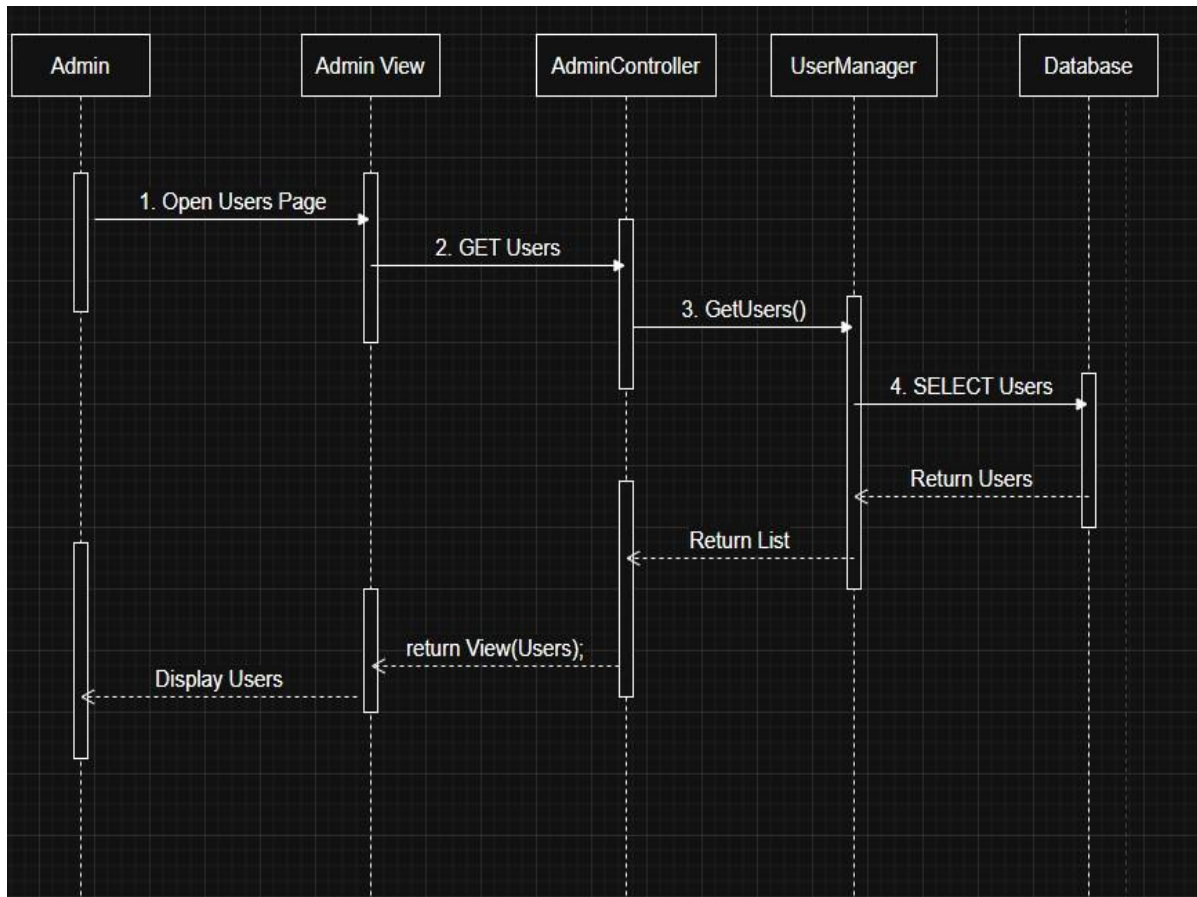
Dijagram sekvenci – Kreiranje turnira

Sekvencni dijagram – Upravljanje korisnicima

Sekvencni dijagram upravljanja korisnicima prikazuje komunikaciju između administratora i sistema tokom pregleda i upravljanja registrovanim korisnicima. Proces započinje otvaranjem stranice sa listom korisnika, nakon čega kontroler pribavlja podatke iz baze i prikazuje ih administratoru.

Administrator zatim bira željenu akciju nad korisničkim nalogom, kao što je promena korisničke uloge ili brisanje korisnika. Prilikom brisanja naloga sistem najpre proverava da li administrator pokušava da obriše sopstveni nalog. Ukoliko je taj uslov ispunjen, operacija se prekida i administrator dobija odgovarajuće obaveštenje.

U slučaju da je izabran drugi korisnik, sistem pronalazi odgovarajući nalog, uklanja ga iz baze podataka i nakon uspešno izvršene operacije prikazuje ažuriranu listu korisnika. Na taj način administrator efikasno upravlja korisničkim nalogima uz poštovanje definisanih poslovnih pravila sistema. Prikazani sekvencni dijagram prikazuje razmenu poruka između administratora, korisničkog interfejsa, kontrolera, Identity servisa i baze podataka tokom procesa upravljanja korisnicima.



Dijagram sekvenci – Upravljanje korisnicima

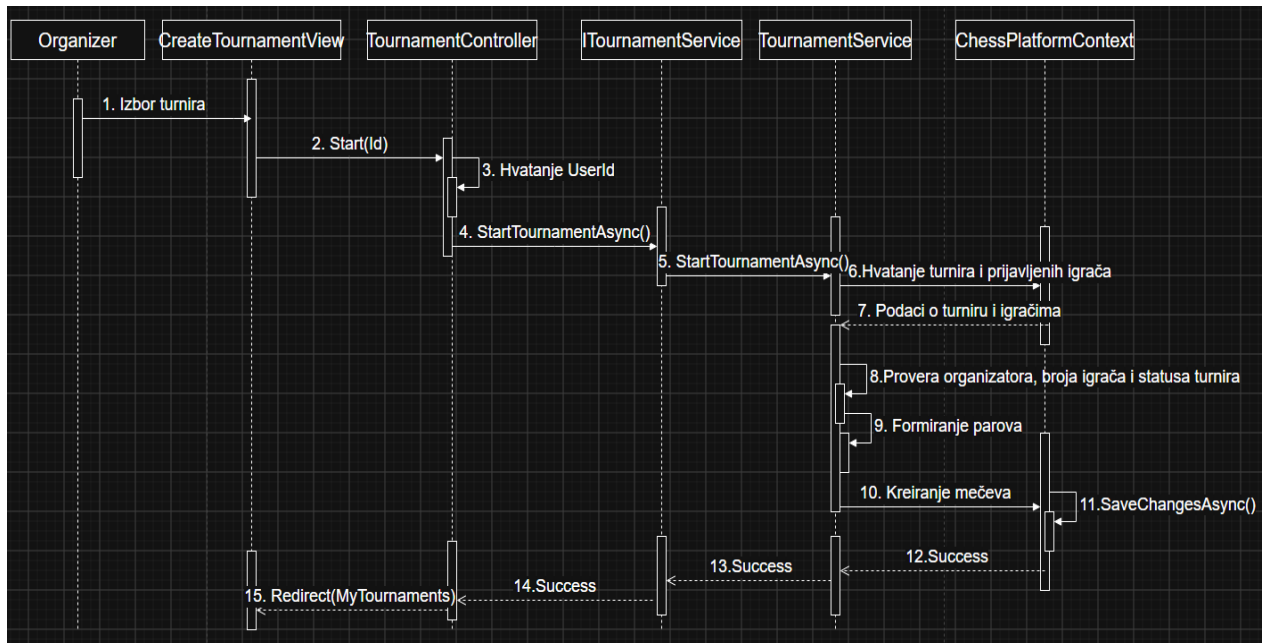
Sekvencni dijagram – Pokretanje turnira

Sekvencni dijagram pokretanja turnira prikazuje tok izvršavanja procesa pokretanja turnira od strane organizatora. Proces započinje kada organizator na stranici svojih turnira izabere opciju za pokretanje odgovarajućeg turnira. Zahtev se prosleđuje kontroleru, koji identifikuje prijavljenog korisnika i prosleđuje zahtev servisnom sloju.

Servisni sloj, odnosno TournamentService, proverava da li je organizator korisnik koji je kreirao izabrani turnir, da li turnir već nije pokrenut i da li postoji dovoljan i paran broj prijavljenih igrača. Ukoliko neki od uslova nije ispunjen, pokretanje se prekida i korisniku se prikazuje odgovarajuća poruka.

Ukoliko su svi uslovi zadovoljeni, servisni sloj formira parove prijavljenih igrača i kreira odgovarajuće mečeve. Podaci o mečevima i promenjenom statusu turnira zatim se čuvaju u bazi podataka putem ChessPlatformContext objekta. Nakon uspešnog završetka operacije, organizatoru se prikazuje ažuriran pregled turnira.

Prikazani sekvencni dijagram ilustruje komunikaciju između organizatora, korisničkog interfejsa, kontrolera, servisnog sloja i konteksta baze podataka tokom procesa pokretanja turnira.



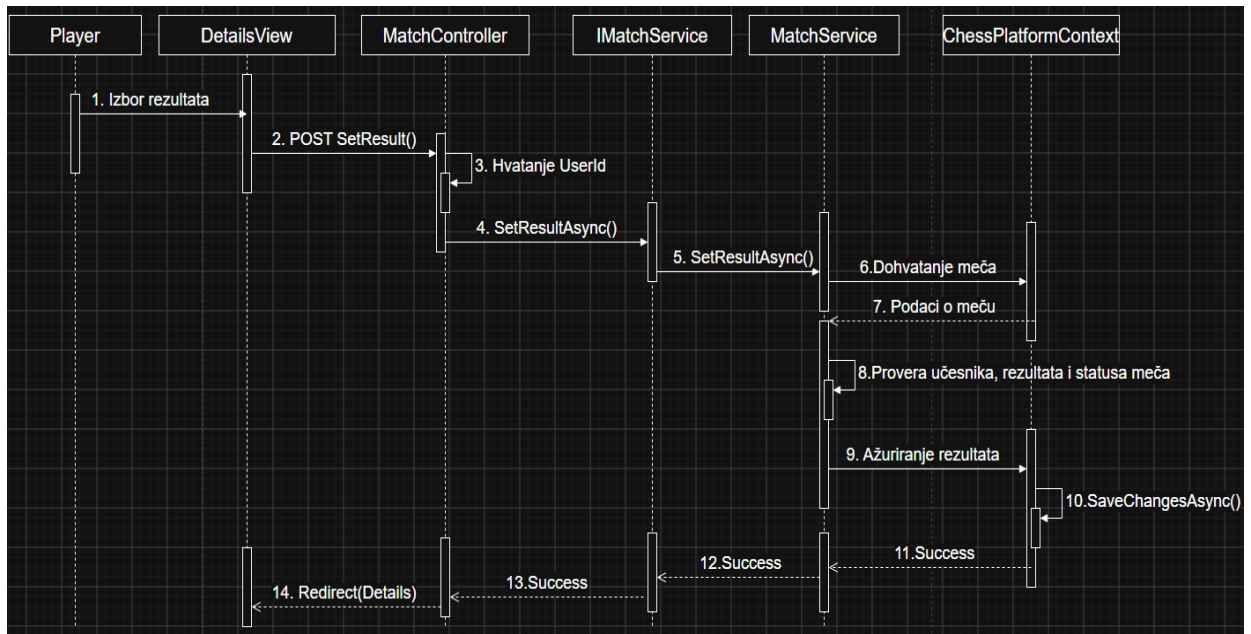
Sekvenčni dijagram – Evidentiranje rezultata meča

Sekvenčni dijagram evidentiranja rezultata meča prikazuje tok izvršavanja procesa unosa rezultata od strane igrača koji učestvuje u meču. Proces započinje kada igrač otvori detalje svog meča i izabere odgovarajući rezultat, odnosno pobjedu belog, pobjedu crnog ili remi.

Nakon slanja zahteva, kontroler prosleđuje podatke servisnom sloju, odnosno interfejsu `IMatchService`, koji poziva implementaciju `MatchService`. Servisni sloj proverava da li meč postoji, da li je igrač jedan od učesnika meča i da li meč prethodno nije završen. Takođe se proverava da li je prosleđeni rezultat dozvoljena vrednost.

Ukoliko su svi uslovi ispunjeni, servisni sloj ažurira podatke o meču, postavlja odgovarajući rezultat i označava meč kao završen. Podaci se zatim čuvaju u bazi podataka putem `ChessPlatformContext` objekta. Nakon uspešnog evidentiranja rezultata, igraču se prikazuje ažurirano stanje meča, a rezultat postaje dostupan za obračun plasmana na turniru.

Ukoliko neki od uslova nije ispunjen, rezultat se ne upisuje i korisniku se prikazuje odgovarajuća poruka. Prikazani sekvenčni dijagram ilustruje komunikaciju između igrača, korisničkog interfejsa, kontrolera, servisnog sloja i konteksta baze podataka tokom procesa evidentiranja rezultata meča.



4.2 Ugovori o sistemskim operacijama

Ugovori o sistemskim operacijama definišu ponašanje sistema prilikom izvršavanja ključnih funkcionalnosti. Za svaku operaciju navode se njen naziv, ulazni parametri, preduslovi, postuslovi i rezultat izvršavanja.

4.2.1 Registracija korisnika

Tabela 3. Ugovor o sistemskim operacijama – Registracija korisnika

Element	Opis
Operacija	Registracija korisnika
Akter	Neregistrovani korisnik
Ulazni parametri	Korisničko ime, ime i prezime, datum rođenja, e-mail adresa, lozinka
Preduslov	Korisnik nije registrovan u sistemu.
Postuslov	Kreiran je novi korisnički nalog sa ulogom Player.

Rezultat	Korisnik može da se prijavi u sistem korišćenjem unetih kredencijala.
----------	---

4.2.2 Kreiranje turnira

Tabela 4. Ugovor o sistemskim operacijama – Kreiranje turnira

Element	Opis
Operacija	Kreiranje turnira
Akter	Organizator
Ulazni parametri	Naziv turnira, opis, tip turnira, vreme, datum početka, datum završetka
Preduslov	Organizator je uspešno prijavljen u sistem.
Postuslov	Novi turnir je sačuvan u bazi podataka.
Rezultat	Turnir je dostupan za prijavu igrača.

4.2.3 Upravljanje korisnicima

Tabela 5. Ugovor o sistemskim operacijama – Upravljanje korisnicima

Element	Opis
Operacija	Upravljanje korisnicima
Akter	Administrator
Ulazni parametri	Identifikator korisnika i željena akcija (promena uloge ili brisanje)
Preduslov	Administrator je prijavljen u sistem.

Postuslov	Korisnički podaci ili korisnička uloga su uspešno ažurirani, odnosno korisnik je obrisani iz sistema ukoliko je operacija dozvoljena.
Rezultat	Lista korisnika je ažurirana u skladu sa izvršenom operacijom.

4.3 Konceptualni model baze podataka

Konceptualni model baze podataka predstavlja logički prikaz strukture sistema i odnosa između osnovnih entiteta bez oslanjanja na fizičku implementaciju baze podataka. Cilj ovog modela jeste da prikaže ključne objekte sistema i njihove međusobne veze na visokom nivou apstrakcije.

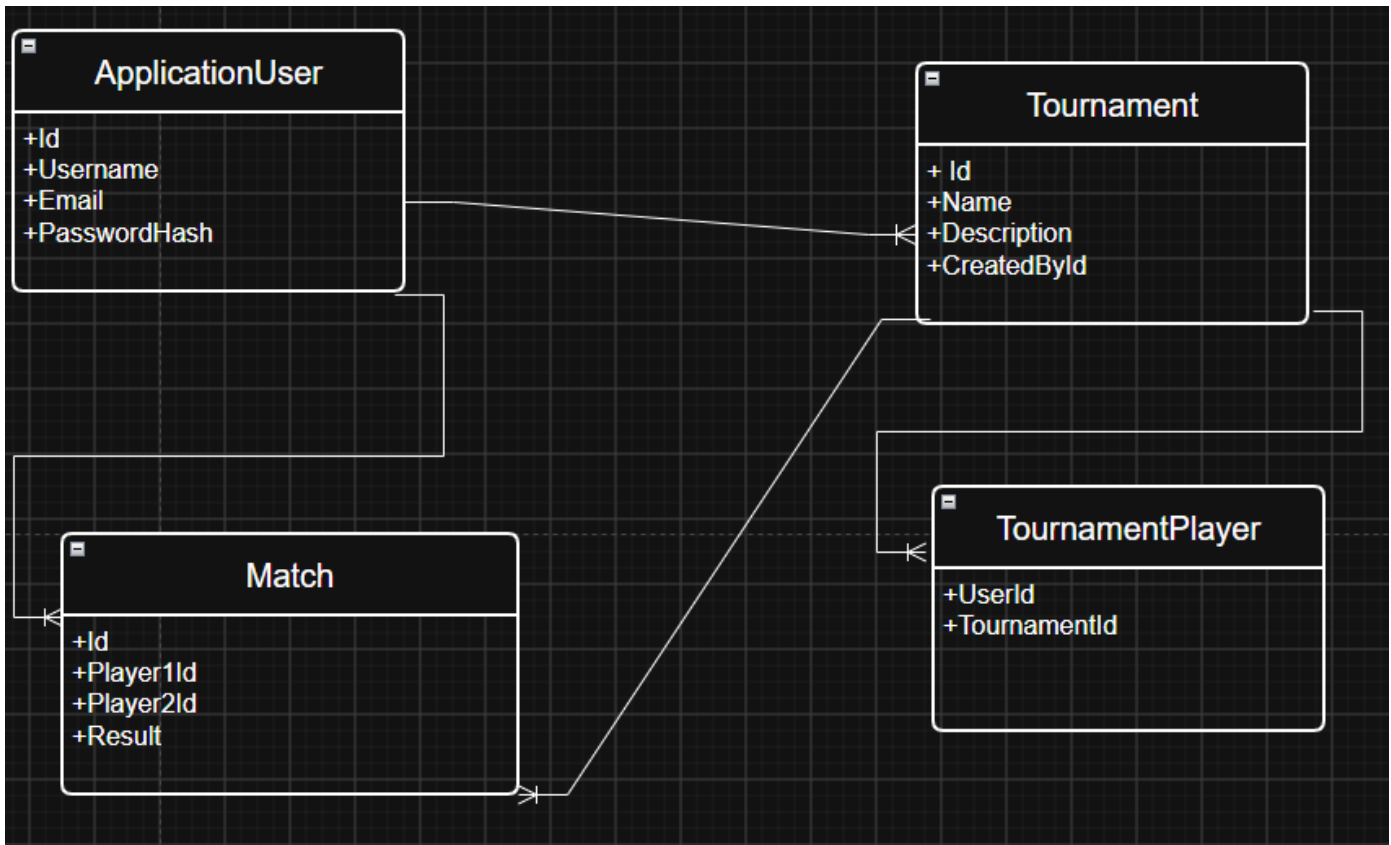
U okviru razvijene aplikacije identifikovani su sledeći osnovni entiteti: korisnik (ApplicationUser), turnir (Tournament), meč (Match) i pomoćni entitet za povezivanje korisnika i turnira (TournamentPlayer). Entitet korisnik predstavlja sve registrovane korisnike sistema, uključujući igrače, organizatore i administratore, pri čemu se njihove uloge definišu kroz sistem autentifikacije i autorizacije.

Entitet turnir predstavlja takmičenje koje kreira organizator i koje sadrži osnovne informacije o turniru, njegovom trajanju, tipu i vremenskoj kontroli. Svaki turnir je povezan sa korisnikom koji ga je kreirao i može sadržati više mečeva, čime se uspostavlja veza jedan-prema-više između turnira i mečeva.

Entitet meč predstavlja pojedinačnu partiju između dva igrača unutar određenog turnira i sadrži informacije o učesnicima, rezultatu i statusu završetka. Svaki meč pripada tačno jednom turniru, dok korisnik može učestvovati u više mečeva, bilo kao prvi ili drugi igrač.

Kako bi se omogućila relacija više-prema-više između korisnika i turnira, uveden je pomoćni entitet TournamentPlayer, koji predstavlja prijavu korisnika na određeni turnir. Jedan korisnik može biti prijavljen na više turnira, dok jedan turnir može imati više prijavljenih korisnika.

Prikazani konceptualni model omogućava jasno razumevanje strukture sistema i odnosa između njegovih osnovnih entiteta i predstavlja osnovu za implementaciju relacionog modela baze podataka.



Konceptualni model sistemskih operacija

4.4 Relacioni model baze podataka

Relacioni model baze podataka predstavlja fizičku implementaciju podataka u sistemu i zasniva se na prethodno definisanom konceptualnom modelu. Podaci su organizovani u relacione tabele koje su međusobno povezane putem primarnih i stranih ključeva.

Glavne tabele u sistemu suAspNetUsers, Tournaments, Matches i TournamentPlayers. TabelaAspNetUsers sadrži podatke o korisnicima sistema i predstavlja osnovu za realizaciju autentifikacije i autorizacije putem ASP.NET Core Identity mehanizma. Tabela Tournaments čuva podatke o turnirima i sadrži referencu na korisnika koji je kreirao turnir. Pored osnovnih podataka, turnir sadrži i informacije o tipu turnira i vremenskoj kontroli.

Tabela Matches predstavlja pojedinačne mečeve između dva korisnika unutar određenog turnira. Učesnici meča određeni su atributima Player1Id i Player2Id, dok se rezultat čuva u atributu Result. Atribut WinnerId omogućava evidentiranje korisnika koji je ostvario pobjedu, dok atribut IsFinished označava da li je meč završen.

Pomoćna tabela TournamentPlayers omogućava realizaciju relacije više-prema-više između korisnika i turnira. Svaki zapis predstavlja prijavu jednog korisnika na određeni turnir, dok se jedinstvenom kombinacijom atributa TournamentId i PlayerId sprečava višestruka prijava istog korisnika na isti turnir.

Relacije između tabela omogućavaju povezivanje korisnika sa turnirima koje kreiraju, mečevima u kojima učestvuju i turnirima na koje su prijavljeni. Na ovaj način obezbeđena je konzistentna struktura baze podataka koja podržava ključne funkcionalnosti sistema.

AspNetUsers(Id, UserName, Email, PasswordHash)

Tournaments(Id, Name, Description, StartDate, EndDate, IsStarted, TournamentType, TimeControlMinutes, CreatedById)

Matches(Id, TournamentId, Player1Id, Player2Id, WinnerId, Result, IsFinished)

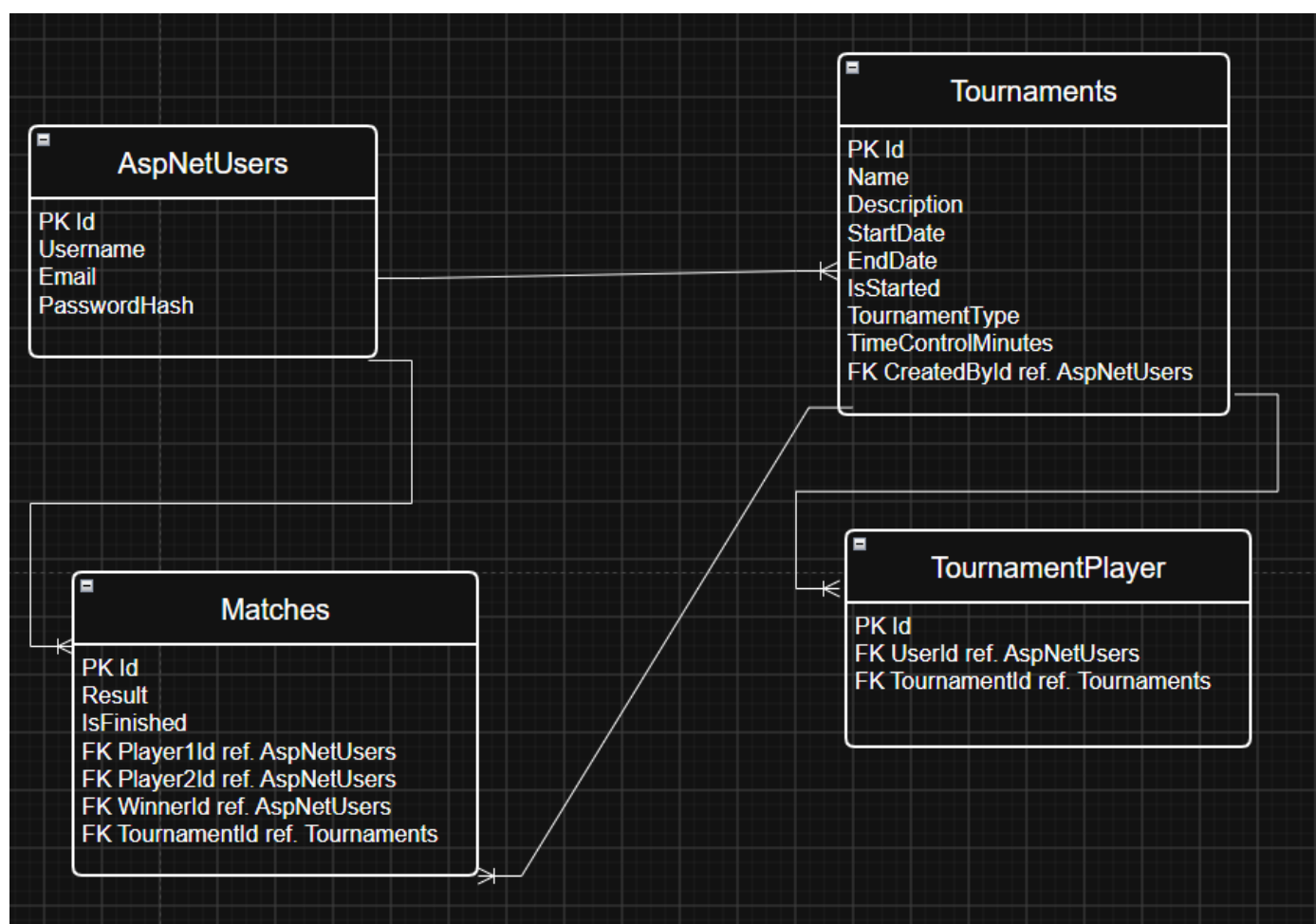
TournamentPlayers(Id, TournamentId, PlayerId)

Relacije između tabela:

- AspNetUsers 1:N Tournaments (CreatedById)
- Tournaments 1:N Matches (TournamentId)
- AspNetUsers 1:N Matches (Player1Id)
- AspNetUsers 1:N Matches (Player2Id)
- AspNetUsers M:N Tournaments preko tabele TournamentPlayers

Prilikom brisanja turnira sistem uklanja i podatke koji su direktno povezani sa tim turnirom, uključujući prijave igrača i mečeve formirane u okviru turnira. Na ovaj način sprečava se zadržavanje podataka koji više nemaju odgovarajući kontekst u sistemu.

Pored toga, na entitetu TournamentPlayer uveden je jedinstveni kompozitni indeks nad kolonama TournamentId i PlayerId. Njegova uloga je da na nivou baze podataka spreči višestruko evidentiranje istog igrača na istom turniru. Time je poslovno pravilo prijavljivanja igrača dodatno zaštićeno i na nivou baze podataka, nezavisno od provera koje se izvršavaju u servisnom sloju.

*Relacioni model baze podataka*

5. Projektovanje poslovne logike

Projektovanje poslovne logike predstavlja fazu razvoja sistema u kojoj se definiše način realizacije funkcionalnih zahteva identifikovanih tokom analize. U ovoj fazi određuju se odgovornosti pojedinih komponenti sistema, njihova međusobna komunikacija, tok izvršavanja sistemskih operacija i način obrade korisničkih zahteva.

Aplikacija je razvijena primenom MVC (Model-View-Controller) arhitekture, pri čemu kontroleri predstavljaju ulaznu tačku za obradu zahteva korisnika, modeli predstavljaju entitete i podatke sistema, dok pogledi omogućavaju prikaz podataka i interakciju sa korisnikom. Za realizaciju poslovne logike uveden je servisni sloj, čime je poslovna logika izdvojena iz kontrolera i omogućeno jasnije razdvajanje odgovornosti između pojedinih slojeva aplikacije.

Servisni sloj obuhvata funkcionalnosti vezane za upravljanje turnirima i mečevima. Servisi obrađuju poslovna pravila sistema, vrše provere podataka i, u saradnji sa Entity Framework Core slojem, izvršavaju operacije nad bazom podataka. Kontroleri su na taj način zaduženi prvenstveno za prijem HTTP zahteva, komunikaciju sa korisnikom i prosleđivanje zahteva odgovarajućem servisu.

Za pristup podacima koristi se Entity Framework Core, pri čemu ChessPlatformContext predstavlja centralnu komponentu za komunikaciju sa relacionom bazom podataka. Funkcionalnosti upravljanja turnirima i mečevima realizuju se kroz servisni sloj.

U nastavku poglavlja prikazana je organizacija poslovne logike, uloga servisnog sloja i način razdvajanja odgovornosti između kontrolera, servisa i sloja za pristup podacima. Takođe su prikazani kolaboracioni dijagrami koji opisuju komunikaciju objekata tokom izvršavanja odabranih sistemskih operacija.

5.1 Arhitektura i organizacija poslovne logike

Aplikacija je organizovana primenom MVC arhitekture, uz dodatno izdvajanje poslovne logike u servisni sloj. Ovakva organizacija omogućava razdvajanje odgovornosti između komponenti sistema i olakšava održavanje i proširivanje aplikacije.

Controller sloj predstavlja ulaznu tačku za HTTP zahteve korisnika. Kontroleri prihvataju podatke iz korisničkog interfejsa, vrše osnovnu proveru modela i prosleđuju zahteve odgovarajućim servisima. Nakon izvršavanja operacije, kontroler korisniku vraća odgovarajući pogled ili izvršava preusmeravanje na drugu akciju.

Service sloj zadužen je za realizaciju poslovne logike sistema. U okviru aplikacije definisani su interfejsi ITournamentService i IMatchService, kao i njihove implementacije TournamentService i MatchService. Servisi obuhvataju operacije kao što su kreiranje, prijavljivanje i odjavljivanje igrača, pokretanje i brisanje turnira, učitavanje podataka o mečevima, unos rezultata i izračunavanje plasmana igrača. Poslovna pravila se izvršavaju unutar servisa, čime se sprečava njihovo nepotrebno ponavljanje u kontrolerima.

Data access sloj realizovan je korišćenjem tehnologije Entity Framework Core. Klasa ChessPlatformContext predstavlja kontekst baze podataka i omogućava rad sa entitetima sistema i izvršavanje operacija nad bazom podataka. Servisi koriste ovaj kontekst za čitanje i izmenu podataka, dok Entity Framework Core upravlja komunikacijom sa relacionom bazom podataka.

Pored navedenih komponenti, projekat je organizovan u više osnovnih direktorijuma. Controllers sadrži kontrolere odgovorne za obradu HTTP zahteva, Models sadrži entitete sistema, Views sadrži Razor prikaze korisničkog interfejsa, Services sadrži interfejse i implementacije servisnog sloja, dok ViewModels sadrži modele namenjene prenosu podataka između kontrolera i pogleda.

Na ovaj način ostvareno je jasno razdvajanje odgovornosti između korisničkog interfejsa, obrade zahteva, poslovne logike i pristupa podacima. Takva organizacija omogućava lakše testiranje pojedinačnih delova sistema, jednostavnije održavanje postojećih funkcionalnosti i dodavanje novih funkcionalnosti bez nepotrebnog menjanja ostalih slojeva aplikacije.

5.2 Servisni sloj

Servisni sloj predstavlja deo aplikacije zadužen za implementaciju poslovne logike sistema. Njegovim uvođenjem poslovna pravila su izdvojena iz kontrolera, čime je ostvareno jasnije razdvajanje odgovornosti i smanjena zavisnost kontrolera od načina realizacije pojedinačnih operacija.

U okviru aplikacije definisana su dva osnovna servisa: TournamentService i MatchService. Za svaki servis definisan je odgovarajući interfejs, ITournamentService i IMatchService, koji određuje operacije dostupne ostatku aplikacije. Servisi se u aplikaciju uključuju korišćenjem mehanizma dependency injection koji obezbeđuje ASP.NET Core.

TournamentService zadužen je za poslovne operacije povezane sa turnirima. Njegove funkcionalnosti obuhvataju kreiranje turnira, prijavljivanje igrača, odjavljivanje igrača, brisanje turnira, pokretanje turnira i izračunavanje tabele plasmana. Prilikom izvršavanja ovih operacija servis proverava poslovna pravila sistema. Na primer, prilikom kreiranja proverava se ispravnost datuma, dok se prilikom prijavljivanja proverava da li turnir postoji, da li je već pokrenut i da li je igrač već prijavljen. Prilikom pokretanja turnira proverava se da li je organizator vlasnik turnira, da li je turnir već pokrenut i da li postoji dovoljan i odgovarajući broj prijavljenih igrača.

MatchService obuhvata poslovnu logiku vezanu za šahovske mečeve. Servis omogućava učitavanje podataka o konkretnom meču uz proveru korisnika koji mu pristupa, kao i evidentiranje rezultata meča. Prilikom unosa rezultata proverava se da li meč postoji, da li korisnik učestvuje u tom meču, da li je meč već završen i da li je prosleđeni rezultat dozvoljen. Kao mogući rezultati podržani su pobeda belog igrača, pobeda crnog igrača i remi.

Servisni sloj koristi ChessPlatformContext za pristup podacima. Na taj način kontroleri nisu direktno odgovorni za izvršavanje poslovnih pravila i većine operacija nad bazom podataka. Kontroler dobija zahtev korisnika, prosleđuje potrebne podatke servisu i na osnovu povratne vrednosti određuje odgovarajući odgovor ili preusmeravanje.

Posebna funkcionalnost TournamentService servisa je izračunavanje tabele plasmana. Na osnovu završenih mečeva određuju se broj odigranih partija, broj pobeda, remija i poraza, kao i ukupan broj osvojenih bodova. Za pobjedu se dodjeljuje jedan bod, za remi 0,5 bodova, dok se za poraz ne dodjeljuju bodovi. Igrači se zatim rangiraju prema osvojenim bodovima, a kao dodatni kriterijum koristi se broj pobjeda.

Ovakvom organizacijom poslovne logike omogućeno je da se postojeće funkcionalnosti lakše održavaju i proširuju. Dodavanje novih poslovnih pravila može se izvršiti u odgovarajućem servisu bez potrebe za prebacivanjem iste logike u više različitih kontrolera.

Tabela 6. Poslovna pravila

Oznaka	Poslovno pravilo
PR-01	Datum završetka turnira mora biti nakon datuma početka turnira.
PR-02	Isti igrač ne može biti prijavljen više puta na isti turnir.
PR-03	Igrač ne može da se prijavi na turnir koji je već pokrenut.
PR-04	Igrač ne može da napusti turnir koji je već pokrenut.
PR-05	Turnir može da pokrene samo organizator koji ga je kreirao.
PR-06	Turnir može da se pokrene samo ako ima najmanje dva prijavljena igrača.
PR-07	Turnir može da se pokrene samo ako je broj prijavljenih igrača paran.
PR-08	Turnir koji je već pokrenut ne može ponovo da se pokrene.
PR-09	Prilikom pokretanja turnira formiraju se mečevi između prijavljenih igrača.
PR-10	Rezultat meča može evidentirati samo igrač koji učestvuje u tom meču.
PR-11	Rezultat se može evidentirati samo za meč koji nije prethodno završen.
PR-12	Dozvoljeni rezultati meča su pobjeda belog, pobjeda crnog i remi.
PR-13	Nakon evidentiranja rezultata meč se označava kao završen.
PR-14	Plasman igrača računa se na osnovu završenih mečeva.
PR-15	Za pobjedu igrač dobija 1 poen, za remi 0,5 poena, a za poraz 0 poena.
PR-16	Prilikom brisanja turnira brišu se i prijave igrača i mečevi povezani sa tim turnirom.

Oznaka	Poslovno pravilo
PR-17	Jedinstveni par TournamentId i PlayerId sprečava višestruku prijavu istog igrača na isti turnir.

5.3 Kolaboracioni dijagrami

Kolaboracioni dijagrami prikazuju međusobnu komunikaciju objekata sistema tokom izvršavanja odabranih sistemskih operacija. Za razliku od sekvencnih dijagrama, koji naglašavaju vremenski redosled poruka, kolaboracioni dijagrami stavljaju akcenat na objekte koji učestvuju u realizaciji operacije i njihove međusobne veze.

U okviru aplikacije odabrane su operacije koje predstavljaju karakteristične tokove rada sistema. Za svaku operaciju prikazani su objekti koji učestvuju u njenom izvršavanju i poruke koje se između njih razmenjuju. Dijagrami obuhvataju registraciju korisnika, kreiranje turnira i upravljanje korisnicima od strane administratora.

Prikazani kolaboracioni dijagrami predstavljaju dopunu prethodno opisanom modelu poslovne logike i omogućavaju jasniji prikaz načina na koji pojedine komponente aplikacije međusobno sarađuju.

Kolaboracioni dijagram – Registracija korisnika

Objekti:

:User :RegisterView :AccountController :UserManager :ChessPlatformContext

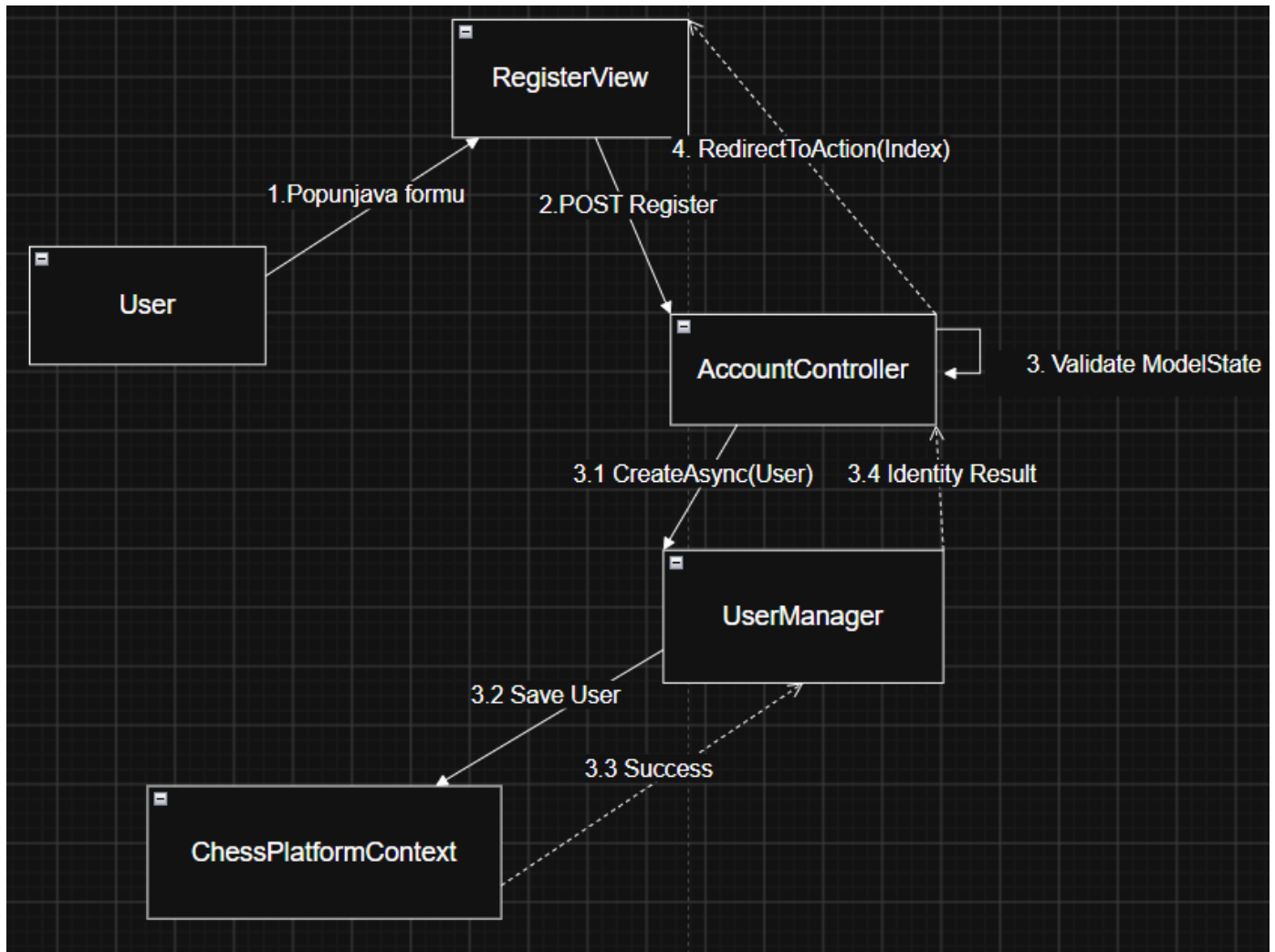
Poruke:

- 1 User → RegisterView : Popunjava formu
- 2 RegisterView → AccountController : POST Register(model)
- 3 AccountController → AccountController : Validate ModelState
 - 3.1 [Valid]
AccountController → UserManager : CreateAsync(user)
 - 3.2 UserManager → ChessPlatformContext : Save User

3.3 ChessPlatformContext → UserManager : Success

3.4 UserManager → AccountController : IdentityResult

4 AccountController → RegisterView : RedirectToAction(Index)



Kolaboracioni dijagram – Registracija korisnika

Kolaboracioni dijagram – Kreiranje turnira

Objekti:

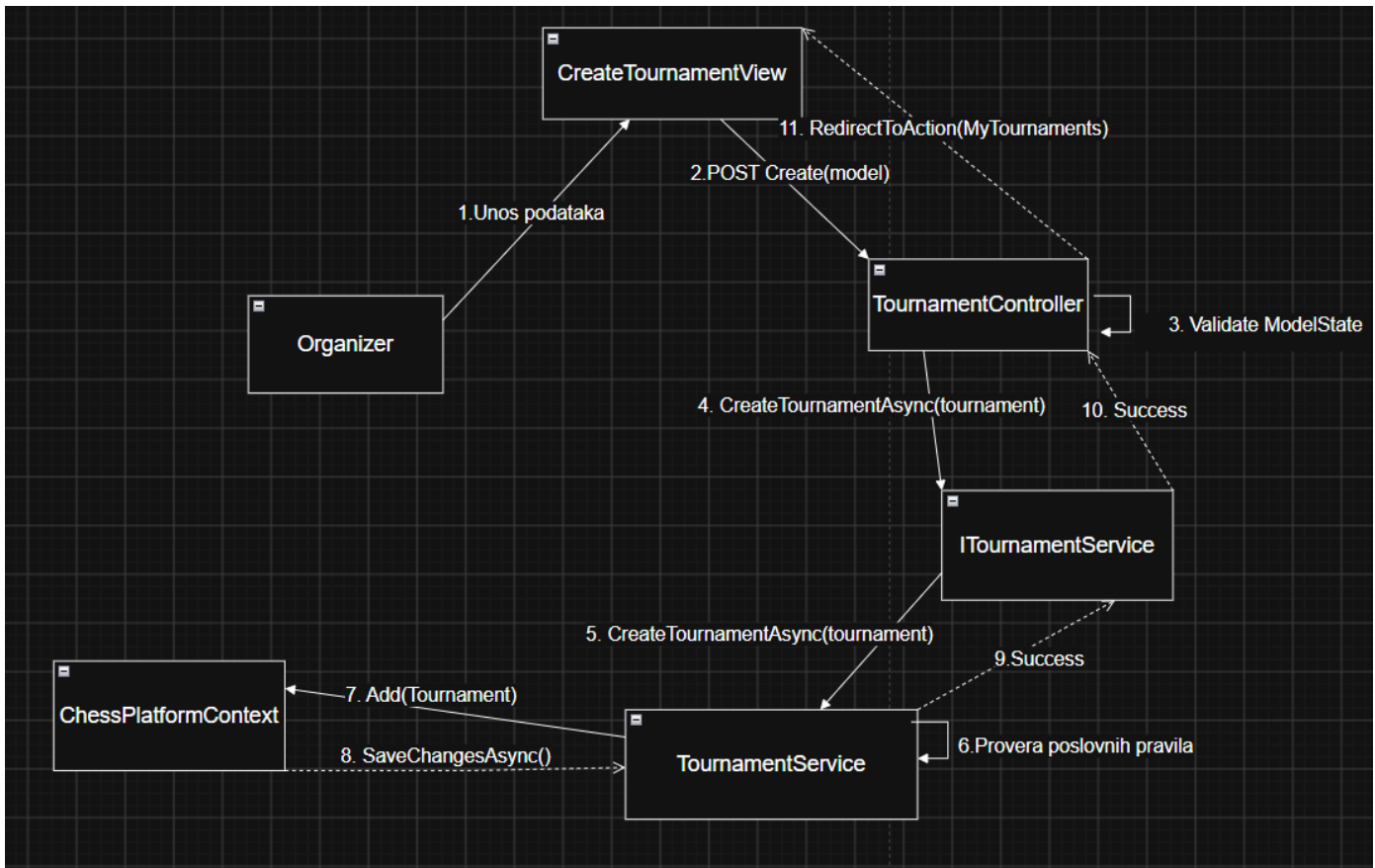
:Organizer

:CreateTournamentView

:TournamentController
:ITournamentService
:TournamentService
:ChessPlatformContext

Poruke:

1. Organizer → CreateTournamentView : Unos podataka
2. CreateTournamentView → TournamentController : POST Create(model)
3. TournamentController → TournamentController : Validate ModelState
4. [Valid] TournamentController → ITournamentService : CreateTournamentAsync(tournament)
5. ITournamentService → TournamentService : CreateTournamentAsync(tournament)
6. TournamentService → TournamentService : Provera poslovnih pravila
7. TournamentService → ChessPlatformContext : Add(tournament)
8. ChessPlatformContext → TournamentService : SaveChangesAsync()
9. TournamentService → ITournamentService : Success
10. ITournamentService → TournamentController : Success
11. TournamentController → CreateTournamentView : Redirect(MyTournaments)



Kolaboracioni dijagram – Kreiranje turnira

Kolaboracioni dijagram – Upravljanje korisnicima

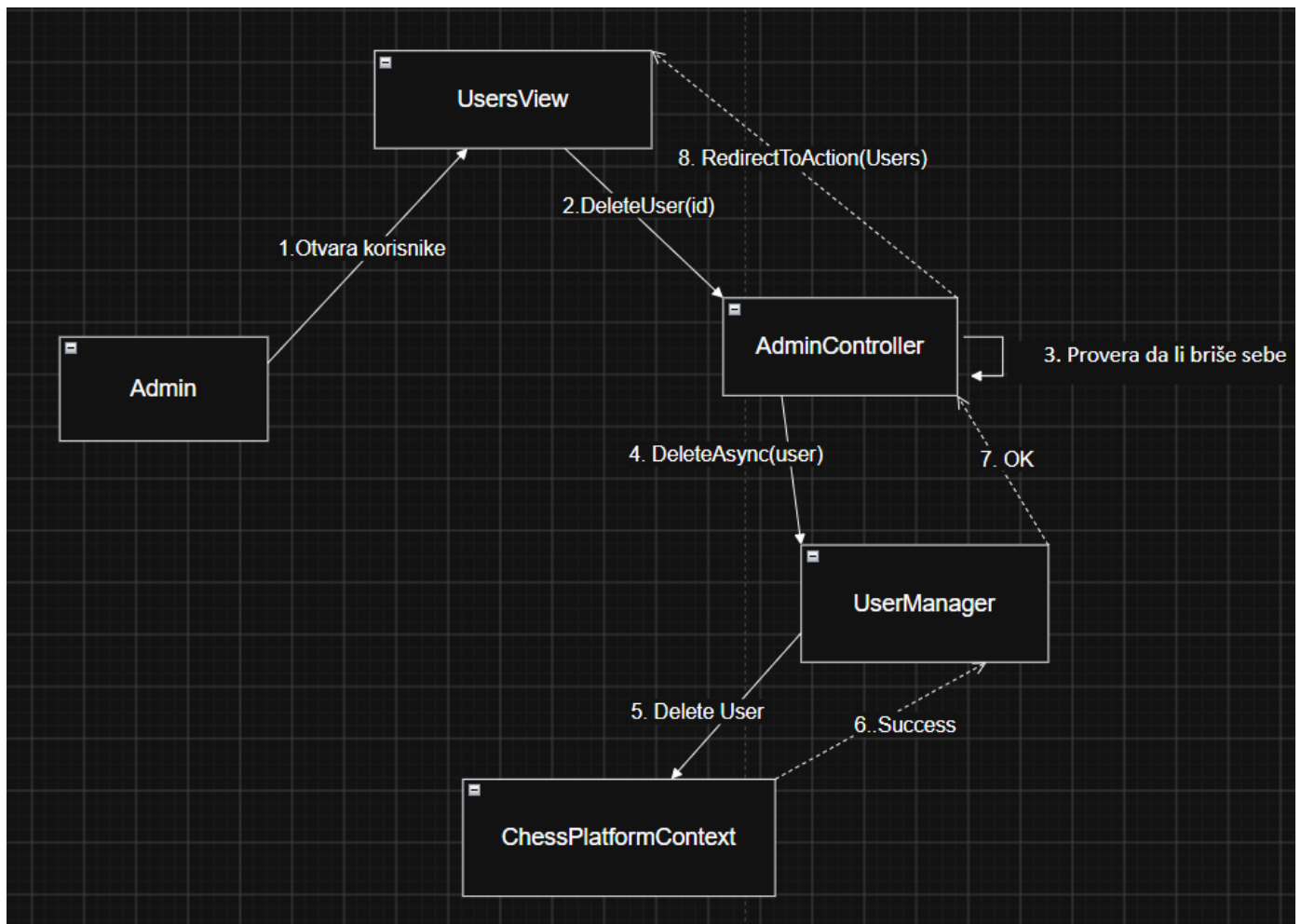
Objekti:

:Admin :UsersView :AdminController :UserManager : ChessPlatformContext Poruke:

- 1 Admin → UsersView : Otvara korisnike
- 2 UsersView → AdminController : DeleteUser(id)
- 3 AdminController → AdminController : Provera da li briše sebe

3.1 [Ne]

- 4 AdminController → UserManager : DeleteAsync(user)
- 5 UserManager → ChessPlatformContext : Delete User
- 6 ChessPlatformContext → UserManager : Success
- 7 UserManager → AdminController : OK
- 8 AdminController → UsersView : RedirectToAction(Users)



Kolaboracioni dijagram – Upravljanje korisnicima

Kolaboracioni dijagram – Pokretanje turnira

Objekti:

:Organizer

:TournamentView

:TournamentController

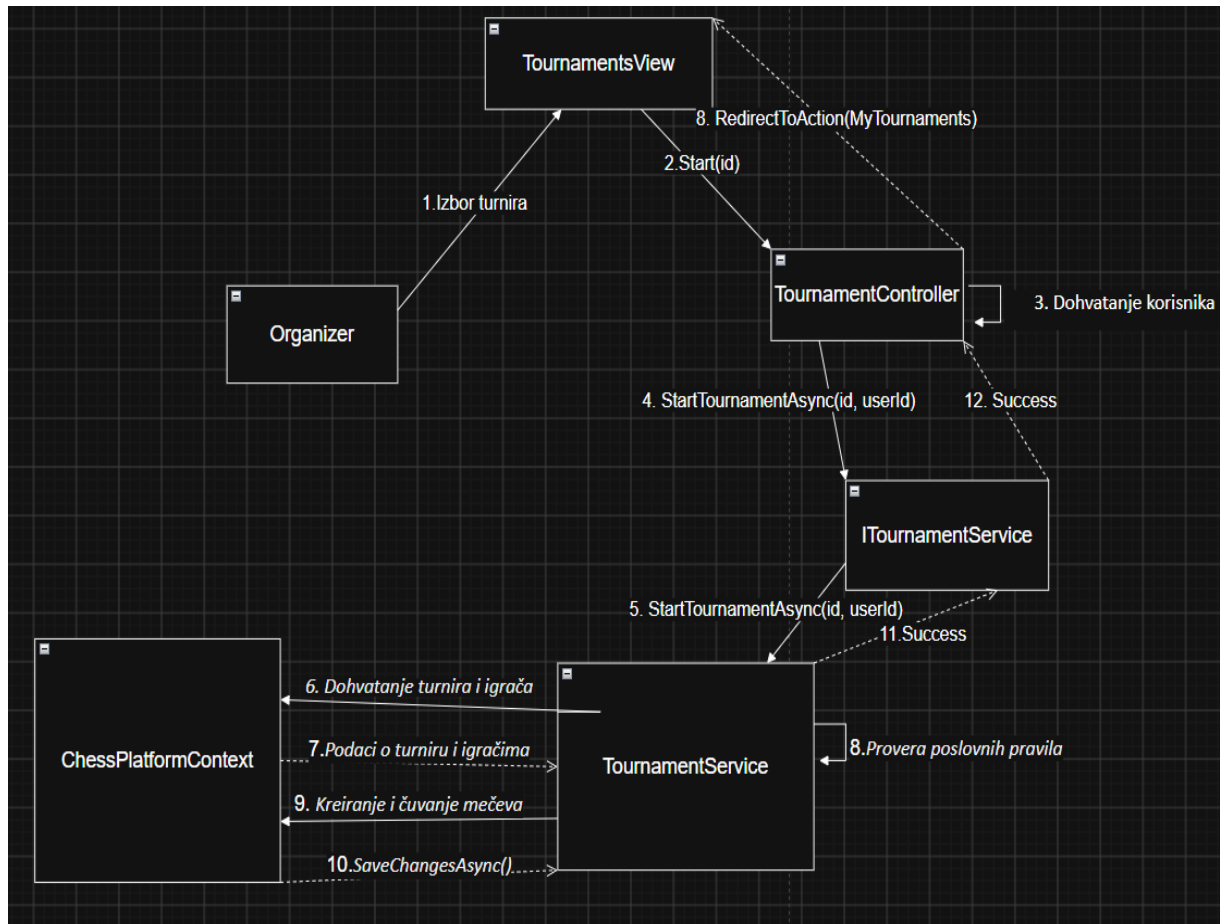
:ITournamentService

:TournamentService

:ChessPlatformContext

Poruke:

1. *Organizer → TournamentView : Izbor turnira*
2. *TournamentView → TournamentController : Start(id)*
3. *TournamentController → TournamentController : Dohvatanje korisnika*
4. *TournamentController → ITournamentService : StartTournamentAsync(id, userId)*
5. *ITournamentService → TournamentService : StartTournamentAsync(id, userId)*
6. *TournamentService → ChessPlatformContext : Dohvatanje turnira i igrača*
7. *ChessPlatformContext → TournamentService : Podaci o turniru i igračima*
8. *TournamentService → TournamentService : Provera poslovnih pravila*
9. *TournamentService → ChessPlatformContext : Kreiranje i čuvanje mečeva*
10. *ChessPlatformContext → TournamentService : SaveChangesAsync()*
11. *TournamentService → ITournamentService : Success*
12. *ITournamentService → TournamentController : Success*
13. *TournamentController → TournamentView : Redirect(MyTournaments)*



Kolaboracioni dijagram – Evidentiranje rezultata meča

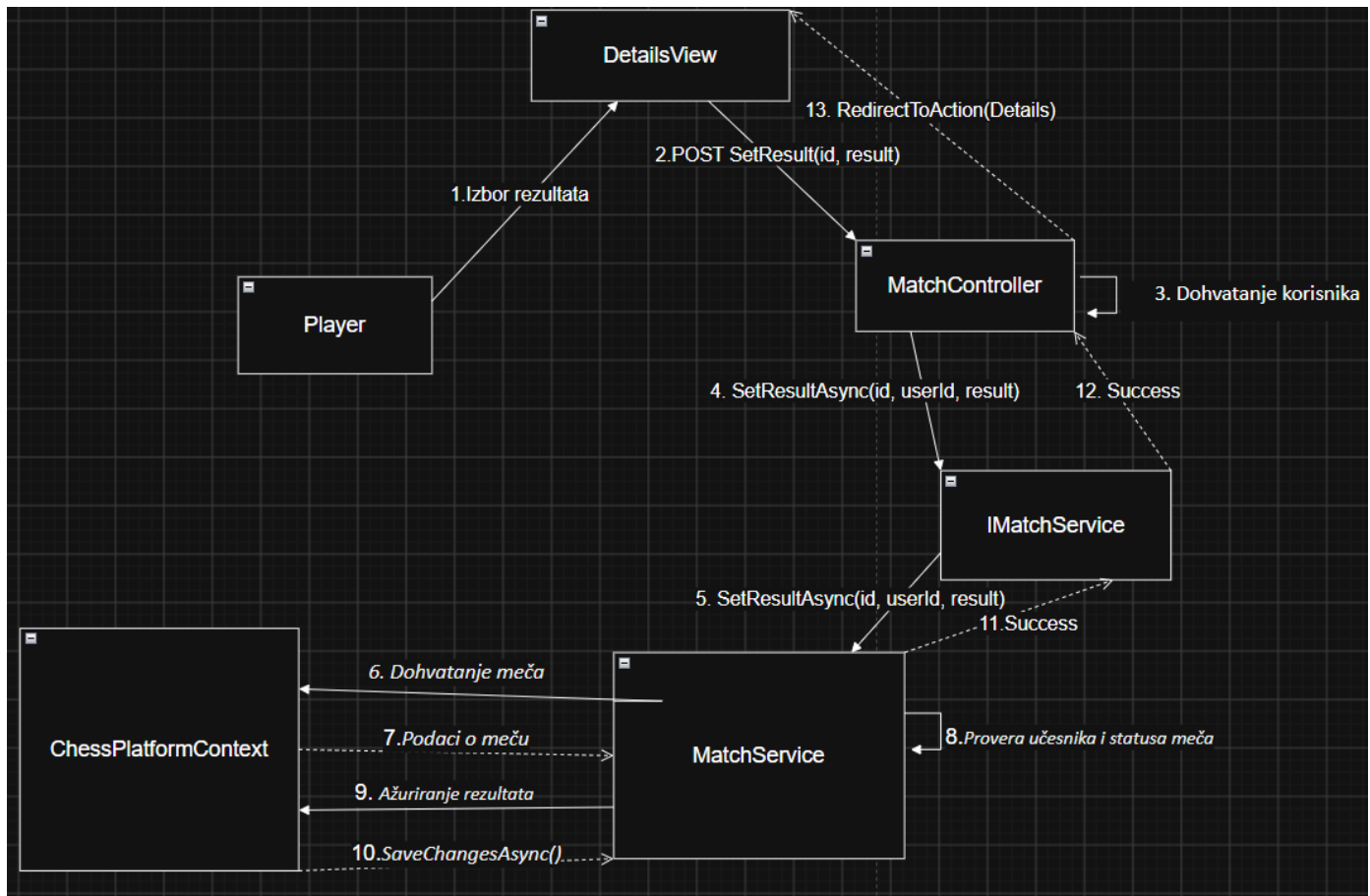
Objekti:

:Player
 :MatchDetailsView
 :MatchController
 :IMatchService
 :MatchService
 :ChessPlatformContext

Poruke:

1. Player → MatchDetailsView : Izbor rezultata
2. MatchDetailsView → MatchController : POST setResult(id, result)
3. MatchController → MatchController : Dohvatanje korisnika
4. MatchController → IMatchService : setResultAsync(id, userId, result)

5. *IMatchService* → *MatchService* : *SetResultAsync(id, userId, result)*
6. *MatchService* → *ChessPlatformContext* : *Dohvatanje meča*
7. *ChessPlatformContext* → *MatchService* : *Podaci o meču*
8. *MatchService* → *MatchService* : *Provera učesnika i statusa meča*
9. [Validno] *MatchService* → *ChessPlatformContext* : *Ažuriranje rezultata*
10. *ChessPlatformContext* → *MatchService* : *SaveChangesAsync()*
11. *MatchService* → *IMatchService* : *Success*
12. *IMatchService* → *MatchController* : *Success*
13. *MatchController* → *MatchDetailsView* : *Redirect(Details)*



6. Implementacija

Implementacija sistema realizovana je korišćenjem razvojnog okruženja Visual Studio 2022 i programskog jezika C#. Aplikacija je razvijena primenom ASP.NET Core MVC okvira, koji omogućava razdvajanje korisničkog interfejsa, obrade zahteva, poslovne logike i pristupa podacima. Za rad sa bazom podataka korišćen je Entity Framework Core kao ORM alat, dok je za autentifikaciju i autorizaciju korisnika korišćen ASP.NET Core Identity.

Tokom implementacije posebna pažnja posvećena je organizaciji aplikacije kroz razdvajanje kontrolera, modela, pogleda, servisnog sloja i sloja za pristup podacima. Na ovaj način ostvareno je jasnije razdvajanje odgovornosti između pojedinačnih komponenti sistema, što olakšava održavanje aplikacije i njeno dalje proširivanje.

U nastavku su prikazani najvažniji delovi implementacije sistema sa odgovarajućim prikazima korisničkog interfejsa i objašnjenjem načina realizacije pojedinačnih funkcionalnosti.

6.1 Implementacija autentifikacije i autorizacije

Autentifikacija i autorizacija korisnika implementirane su korišćenjem ASP.NET Core Identity okvira, koji obezbeđuje upravljanje korisničkim nalogima, prijavom na sistem i kontrolom pristupa funkcionalnostima aplikacije.

Prilikom registracije korisnik unosi korisničko ime, ime i prezime, datum rođenja, e-mail adresu i lozinku putem registracione forme. Nakon validacije unetih podataka kreira se novi korisnički nalog koji se čuva u bazi podataka. Lozinke se ne skladište u izvornom obliku, već se obrađuju korišćenjem mehanizama koje obezbeđuje ASP.NET Core Identity.

Proces prijave realizovan je proverom korisničkih kredencijala, nakon čega sistem uspostavlja autentifikovanu sesiju korisnika. U slučaju neuspešne prijave korisniku se prikazuje odgovarajuća poruka o grešci, dok se nakon uspešne autentifikacije korisnik preusmerava na početnu stranicu odgovarajuću njegovoj ulozi.

Autorizacija je realizovana primenom sistema korisničkih uloga. Aplikacija podržava tri osnovne uloge: Administrator, Organizer i Player. Svakoj ulozi dodeljen je odgovarajući skup funkcionalnosti, čime je omogućeno ograničavanje pristupa pojedinim delovima sistema.



The image shows the 'Register' form on the ChessPlatform website. The form is centered on a background featuring two knights in armor, one in silver with a red cape and the other in black with a black cape. The form has a light beige background with a faint watermark of a chess knight. The fields are as follows:

- Username:
- Password:
- ConfirmedPassword:
- Email:
- FirstName:
- LastName:
- DateOfBirth: (format: mm/dd/yyyy)

At the bottom of the form is a 'Register' button and a link: 'Already have an account? [Login](#)'.

Slika 1. Forma za registraciju korisnika



The image shows the 'Login' form on the ChessPlatform website. The form is centered on the same background as the register form, featuring two knights in armor. The form has a light beige background with a faint watermark of a chess knight. The fields are as follows:

- Username:
- Password:

At the bottom of the form is a 'Login' button and a link: 'Don't have an account? [Register](#)'.

Slika 2. Forma za prijavu korisnika

6.2 Implementacija upravljanja turnirima

Upravljanje turnirima predstavlja jednu od osnovnih funkcionalnosti aplikacije i namenjeno je korisnicima sa ulogom organizatora. Organizator ima mogućnost kreiranja novih turnira, pregleda turnira koje je kreirao, pokretanja turnira i brisanja sopstvenih turnira.

Prilikom kreiranja turnira organizator unosi naziv, opis, tip turnira, vremensku kontrolu, datum početka i datum završetka. Nakon validacije podataka, novi turnir se čuva u bazi podataka. Sistem automatski evidentira korisnika koji je kreirao turnir, čime se omogućava provera vlasništva prilikom izvršavanja pojedinih organizatorskih operacija.

Prilikom kreiranja proverava se da li je datum završetka nakon datuma početka. Ukoliko uslov nije ispunjen, turnir se ne kreira i korisniku se prikazuje odgovarajuća poruka o grešci.

Nakon kreiranja, organizator može pregledati listu svojih turnira. Za svaki turnir prikazane su osnovne informacije, uključujući naziv, opis, tip, vremensku kontrolu i period održavanja. U zavisnosti od trenutnog statusa turnira, organizatoru su dostupne odgovarajuće akcije.

Igrači imaju pristup listi dostupnih turnira. Za svaki turnir prikazane su osnovne informacije i mogućnost prijavljivanja ili napuštanja turnira. Sistem vodi evidenciju prijavljenih igrača i sprečava višestruku prijavu istog korisnika na isti turnir. Prijavljivanje i odjavljivanje nisu dozvoljeni nakon pokretanja turnira.

Prilikom pokretanja turnira sistem proverava da li je zahtev izvršio organizator koji je kreirao dati turnir, da li turnir već nije pokrenut i da li je prijavljen odgovarajući broj igrača. U trenutnoj implementaciji turnir se može pokrenuti kada postoje najmanje dva igrača i kada je njihov broj paran. Nakon uspešne provere sistem formira početne parove i kreira odgovarajuće mečeve.

Create Tournament
Set up your tournament and choose its format.

Tournament Name
Enter tournament name

Description
Describe your tournament

Tournament Type
Select tournament type

Time Control
Select time control

Start Date **End Date**
mm/dd/yyyy mm/dd/yyyy

Create Tournament

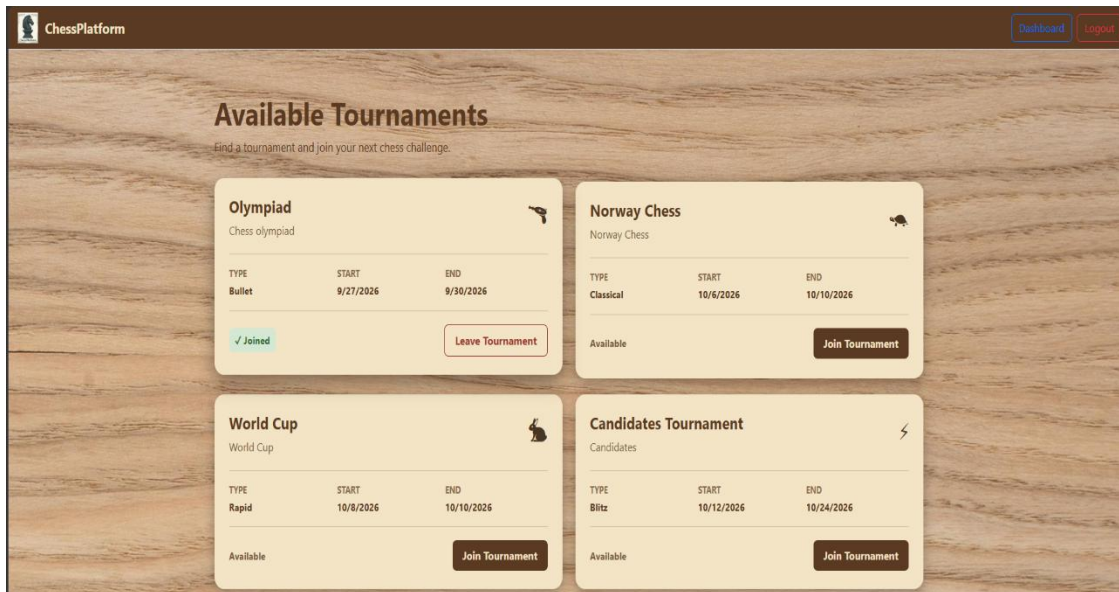
Slika 3. Forma za kreiranje turnira

ChessPlatform Dashboard Logout

My Tournaments
Manage your tournaments and track their progress. + Create Tournament

Name	Description	Tournament Type	Start	End	Status	Actions
Olympiad	Chess olympiad	Bullet	9/27/2026	9/30/2026	Started	Standings Delete
Norway Chess	Norway Chess	Classical	10/6/2026	10/10/2026	Not started	Start Standings Delete
World Cup	World Cup	Rapid	10/8/2026	10/10/2026	Not started	Start Standings Delete
Candidates Tournament	Candidates	Blitz	10/12/2026	10/24/2026	Not started	Start Standings Delete
FreeStyle Championships	FreeStyle	Classical	10/13/2026	10/15/2026	Not started	Start Standings Delete
European Club Cup	Club Cup	Rapid	10/8/2026	10/10/2026	Not started	Start Standings Delete

Slika 4. Prikaz turnira organizatora



Slika 5. Prikaz dostupnih turnira igračima

6.3 Implementacija upravljanja mečevima

Nakon uspešnog pokretanja turnira sistem formira početne parove prijavljenih igrača i kreira odgovarajuće mečeve. Svaki meč sadrži informacije o turniru, učesnicima, trenutnom statusu i rezultatu. Na ovaj način omogućeno je praćenje mečeva u okviru pokrenutog turnira.

Igrači imaju pristup stranici My Matches, na kojoj je prikazana lista mečeva u kojima učestvuju. Za svaki meč prikazani su protivnik, boja figura koju igrač ima, status meča i rezultat. Iz liste mečeva korisnik može otvoriti detaljan prikaz izabranog meča.

Na stranici sa detaljima meča prikazane su informacije o oba učesnika, statusu meča, rezultatu i vremenskoj kontroli definisanoj za turnir. Ukoliko meč još nije završen, učesniku je omogućeno da unese konačan ishod partije izborom jedne od ponuđenih opcija: pobeda belog igrača, pobeda crnog igrača ili remi.

Nakon potvrde rezultata sistem određuje pobednika ukoliko je rezultat pobeda jednog od igrača, čuva rezultat u bazi podataka i označava meč kao završen. Završen rezultat se zatim koristi prilikom izračunavanja plasmana učesnika.

Slika 6. Prikaz mečeva igrača



Slika 7. Prikaz detalja meča

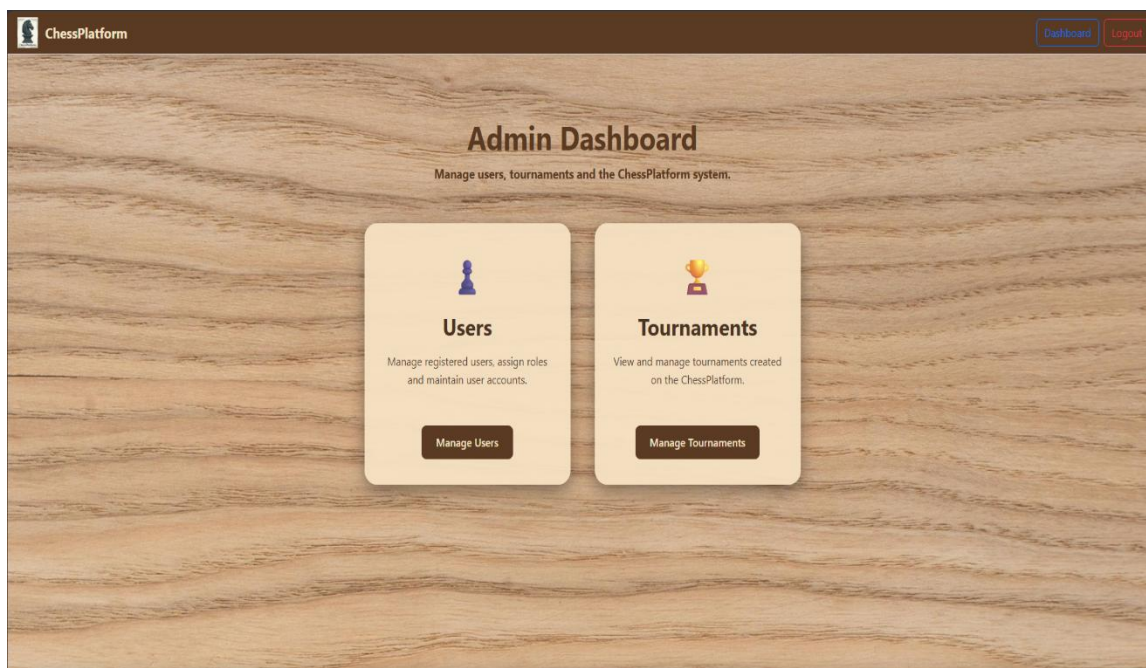
6.4 Implementacija administratorskih funkcionalnosti

Administratorski deo sistema omogućava upravljanje korisnicima i turnirima, kao i kontrolu nad podacima sistema. Ove funkcionalnosti dostupne su isključivo korisnicima sa ulogom Administrator.

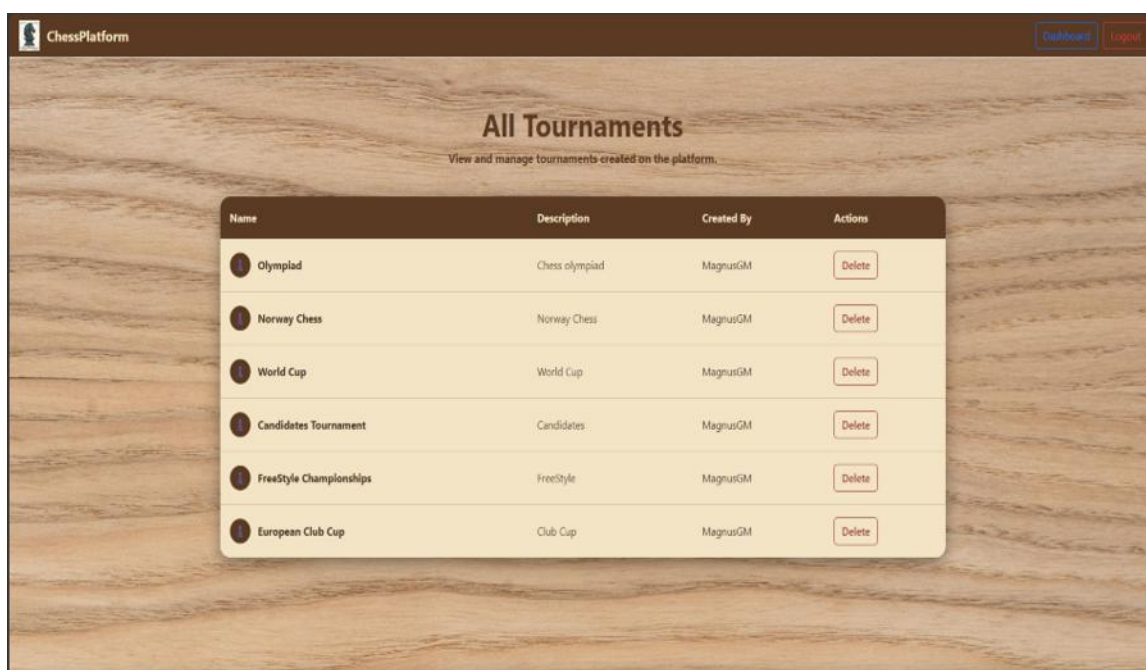
U okviru administratorskog dashboard-a implementirane su opcije za pregled registrovanih korisnika, promenu njihovih korisničkih uloga i brisanje korisničkih naloga. Prilikom brisanja administratora sistem proverava da li administrator pokušava da obriše sopstveni nalog. Takva operacija nije dozvoljena.

Administratori takođe imaju uvid u kreirane turnire, bez obzira na to koji korisnik ih je kreirao. Na taj način administrator može pregledati stanje turnira i izvršiti dozvoljene administratorske operacije nad njima, uključujući njihovo brisanje.

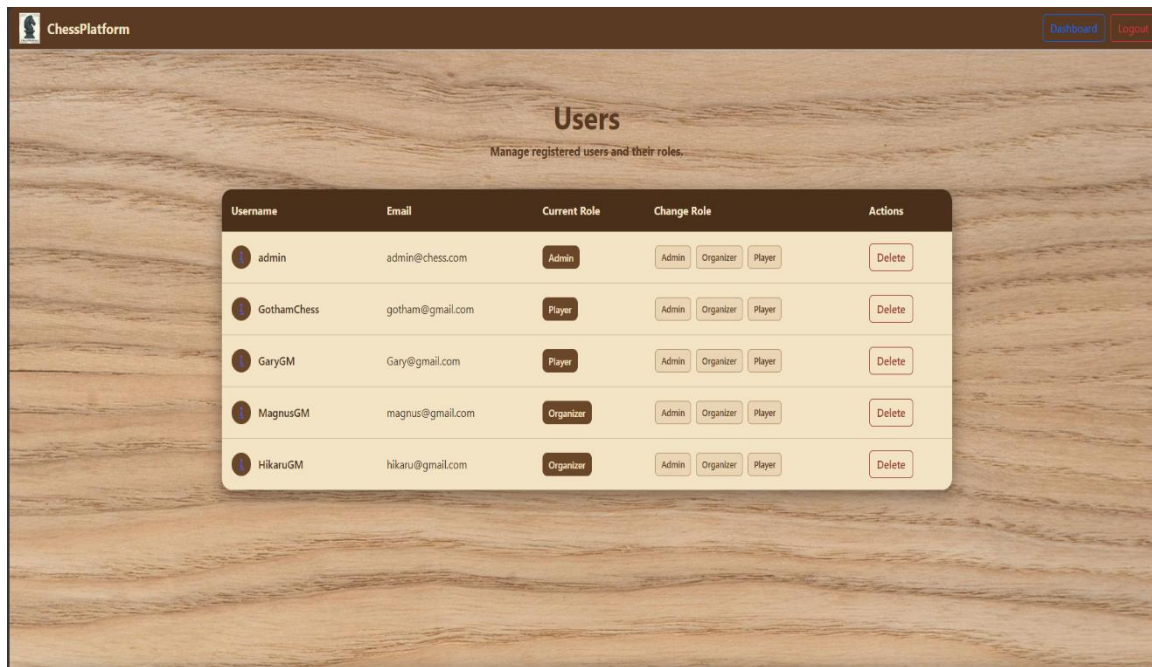
Administratorske funkcionalnosti predstavljaju poseban deo sistema jer omogućavaju upravljanje podacima koji nisu dostupni korisnicima sa ulogama organizatora i igrača. Pristup ovim funkcionalnostima ograničen je sistemom korisničkih uloga.



Slika 8. Početna strana administratora



Slika 9. Prikaz svih kreiranih turnira administratoru



Slika 10. Prikaz svih korisnika

6.5 Implementacija statistike i tabele plasmana

U okviru aplikacije implementiran je prikaz statistike i plasmana igrača na turniru. Podaci se izračunavaju na osnovu prijavljenih igrača i rezultata završenih mečeva.

Za prikaz podataka definisan je TournamentStandingViewModel, koji sadrži ime igrača, broj odigranih mečeva, broj pobeda, remija i poraza, kao i ukupan broj osvojenih poena. Podaci se obrađuju u okviru TournamentService servisa, čime se logika izračunavanja plasmana odvaja od kontrolera i korisničkog interfejsa.

Broj odigranih mečeva određuje se na osnovu završenih mečeva u kojima je igrač učestvovao. Broj pobeda određuje se na osnovu evidentiranog pobjednika, dok se remiji određuju na osnovu rezultata meča. Broj poraza dobija se na osnovu ukupnog broja odigranih mečeva umanjeno za broj pobeda i remija.

Sistem koristi standardni način bodovanja u kome pobeda donosi jedan bod, remi 0,5 bodova, a poraz nula bodova. Nakon izračunavanja statistike, igrači se rangiraju prvo prema ukupnom broju bodova, a zatim prema broju ostvarenih pobeda.

Za prenos podataka iz servisnog sloja ka pogledu koristi se TournamentStandingViewModel. Na taj način pogled dobija samo podatke potrebne za prikaz tabele, bez direktnog pristupa bazi podataka ili poslovnoj logici.

 ChessPlatform

[Dashboard](#) [Logout](#)



Tournament Standings

Current ranking and performance of tournament players:

#	PLAYER	GAMES	WINS	DRAWS	LOSSES	POINTS
1	 GothamChess	0	0	0	0	0
2	 Sindarov	0	0	0	0	0
3	 GaryGM	0	0	0	0	0
4	 Wesley	0	0	0	0	0
5	 FabianoGM	0	0	0	0	0
6	 HansGM	0	0	0	0	0

 ChessPlatform

Chess tournament management system

© 2026 ChessPlatform

7. Ručno testiranje

Ručno testiranje aplikacije sprovedeno je kroz izvršavanje karakterističnih korisničkih scenarija za različite uloge u sistemu. Za svaki scenario provereno je da li aplikacija reaguje u skladu sa definisanim poslovnim pravilima i očekivanim ponašanjem. Testiranje je obuhvatilo autentifikaciju, upravljanje korisnicima, kreiranje i upravljanje turnirima, prijavljivanje igrača, upravljanje mečevima i prikaz plasmana.

Tabela 7. Analiza ručnog testiranja

ID	FUNKCIONALNOST	KORISNIČKA ULOGA	TEST SCENARIO	OČEKIVANI REZULTAT	REZULTAT
T01	Registracija	Gost	Unos ispravnih podataka i slanje registracione forme	Korisnik se uspešno registruje i preusmerava na prijavu	Prošao
T02	Prijava	Gost	Unos ispravnog korisničkog imena i lozinke	Korisnik se uspešno prijavljuje i dobija odgovarajući prikaz prema ulozi	Prošao
T03	Neuspešna prijava	Gost	Unos neispravnih podataka za prijavu	Prijava nije dozvoljena i prikazuje se odgovarajuća poruka	Prošao
T04	Kreiranje turnira	Organizator	Unos ispravnih podataka i kreiranje turnira	Turnir se uspešno kreira i prikazuje na listi turnira	Prošao
T05	Validacija datuma	Organizator	Unos datuma završetka koji je pre ili jednak datumu početka	Kreiranje turnira nije dozvoljeno i prikazuje se poruka o grešci	Prošao
T06	Prijava na turnir	Igrač	Prijavljivanje igrača na postojeći turnir	Igrač se uspešno prijavljuje i turnir se prikazuje među njegovim turnirima	Prošao
T07	Napuštanje turnira	Igrač	Napuštanje turnira pre početka	Igrač se uklanja sa liste učesnika	Prošao
T08	Pokretanje turnira	Organizator	Pokretanje turnira sa najmanje dva učesnika	Turnir se označava kao pokrenut i kreiraju se mečevi	Prošao
T09	Nedovoljan broj igrača	Organizator	Pokušaj pokretanja turnira sa manje od dva igrača	Pokretanje turnira nije dozvoljeno	Prošao

T10	Neparan broj igrača	Organizator	Pokušaj pokretanja turnira sa neparnim brojem igrača	Pokretanje turnira nije dozvoljeno	Prošao
T11	Brisanje turnira	Organizator	Brisanje turnira koji je kreirao prijavljeni organizator	Turnir i povezani podaci se uklanjaju	Prošao
T12	Prikaz meča	Igrač	Otvaranje meča u kojem korisnik učestvuje	Prikazuju se podaci o meču i učesnicima	Prošao
T13	Unos rezultata	Igrač	Izbor rezultata „White Win“	Rezultat se evidentira i meč se označava kao završen	Prošao
T14	Unos rezultata	Igrač	Izbor rezultata „Black Win“	Rezultat se evidentira i pobjednik se pravilno određuje	Prošao
T15	Remi	Igrač	Izbor rezultata „Draw“	Meč se označava kao završen bez pobjednika	Prošao
T16	Izmena završenog meča	Igrač	Pokušaj ponovnog unosa rezultata završenog meča	Izmena rezultata nije dozvoljena	Prošao
T17	Tabela plasmana	Korisnik	Otvaranje plasmana turnira	Prikazuje se tabela sa brojem partija, pobjedama, remijima, porazima i poenima	Prošao
T18	Administracija korisnika	Administrator	Pregled i upravljanje korisnicima	Administrator može da izvršava predviđene operacije nad korisnicima	Prošao

Na osnovu sprovedenog ručnog testiranja provereno je ponašanje aplikacije u karakterističnim scenarijima koji obuhvataju različite korisničke uloge i glavne funkcionalnosti sistema. Posebna pažnja posvećena je proveru poslovnih pravila koja ograničavaju određene operacije, kao što su validacija datuma turnira, uslovi za pokretanje turnira i ograničavanje izmena rezultata nakon završetka meča.

Testirani su i osnovni tokovi rada kroz sistem, počevši od registracije i prijave korisnika, preko kreiranja i prijavljivanja na turnire, pa sve do pokretanja turnira, formiranja mečeva, evidentiranja rezultata i prikaza plasmana. Na ovaj način provereno je da pojedinačne funkcionalnosti pravilno funkcionišu i u okviru šireg toka korišćenja aplikacije.

Rezultati testiranja prikazani u tabeli pokazuju da aplikacija u testiranim scenarijima reaguje u skladu sa očekivanim ponašanjem. Provere su obuhvatile kako uspešne scenarije, tako i situacije u kojima sistem treba da odbije nedozvoljenu operaciju. Time je potvrđeno da su implementirana poslovna pravila

primenjena na odgovarajućim mestima i da korisnici dobijaju očekivane rezultate prilikom korišćenja sistema.

Ručno testiranje je takođe omogućilo proveru povezanosti različitih delova aplikacije. Na primer, nakon pokretanja turnira generisani mečevi mogu biti dostupni učesnicima, unos rezultata menja stanje meča, a završeni rezultati se koriste za izračunavanje podataka prikazanih u tabeli plasmana. Na taj način je provereno ne samo funkcionisanje pojedinačnih funkcionalnosti, već i njihova međusobna povezanost.

8. Zaključak

U okviru ovog rada razvijena je web aplikacija za organizaciju i upravljanje šahovskim turnirima. Cilj rada bio je razvoj sistema koji omogućava jednostavnije upravljanje turnirima, učesnicima i mečevima, uz jasno definisane uloge korisnika i odgovarajuća pravila poslovne logike.

Aplikacija je realizovana primenom ASP.NET Core MVC arhitekture, uz korišćenje Entity Framework Core tehnologije za rad sa relacionom bazom podataka i ASP.NET Core Identity sistema za autentifikaciju i autorizaciju korisnika. Sistem podržava različite uloge korisnika, među kojima su administrator, organizator i igrač, pri čemu svaka uloga ima odgovarajući skup funkcionalnosti prilagođen njenim odgovornostima.

Posebna pažnja posvećena je organizaciji poslovne logike. Uvođenjem servisnog sloja funkcionalnosti vezane za upravljanje turnirima i mečevima izdvojene su iz kontrolera i organizovane u odgovarajuće servise. Na taj način ostvareno je jasnije razdvajanje odgovornosti između korisničkog interfejsa, obrade zahteva, poslovne logike i sloja za pristup podacima. U okviru sistema implementirane su funkcionalnosti kreiranja, pokretanja i brisanja turnira, prijavljivanja i odjavljivanja igrača, formiranja mečeva, evidentiranja rezultata i prikaza plasmana učesnika.

Sistem omogućava organizatoru da prilikom kreiranja turnira definiše osnovne podatke, tip turnira i vremensku kontrolu. Igračima je omogućeno pregledanje dostupnih turnira, prijavljivanje i odjavljivanje pre početka turnira, kao i pregled sopstvenih mečeva. Nakon pokretanja turnira sistem formira mečeve prijavljenih učesnika, dok se rezultati evidentiraju kroz predviđene ishode pobede belog igrača, pobede crnog igrača i remija. Na osnovu završenih mečeva automatski se izračunavaju statistički podaci i formira tabela plasmana.

Tokom razvoja posebna pažnja posvećena je proverbi poslovnih pravila. Implementirane su provere ispravnosti datuma turnira, sprečavanja višestrukog prijavljivanja istog igrača, ograničavanja prijavljivanja i odjavljivanja nakon pokretanja turnira, provere vlasništva nad turnirom prilikom njegovog pokretanja ili brisanja, kao i provere minimalnog i odgovarajućeg broja učesnika potrebnog za pokretanje turnira. Kod upravljanja mečevima dodatno se proverava da li korisnik učestvuje u konkretnom meču, da li je meč već završen i da li je prosleđeni rezultat dozvoljen.

U okviru razvoja posebna pažnja posvećena je i organizaciji baze podataka. Entiteti sistema povezani su odgovarajućim relacijama, dok su za sprečavanje višestrukog prijavljivanja istog igrača na isti turnir uvedena odgovarajuća ograničenja na nivou baze podataka. Korišćenjem Entity Framework Core Code First pristupa promene strukture baze podataka praćene su kroz migracije, čime je omogućeno kontrolisano održavanje strukture baze tokom razvoja aplikacije.

Korisnički interfejs je dodatno prilagođen karakteru aplikacije kroz vizuelni identitet zasnovan na motivima šaha i odgovarajućoj paleti boja. Različite stranice sistema organizovane su tako da korisniku jasno prikažu dostupne funkcionalnosti, podatke o turnirima, mečevima i plasmanu. Posebna pažnja posvećena je responzivnosti interfejsa i prilagođavanju prikaza različitim veličinama ekrana.

Ručno testiranje sprovedeno je kroz karakteristične korisničke scenarije za različite uloge u sistemu. Testirani su osnovni tokovi registracije i prijave korisnika, upravljanje turnirima, prijavljivanje i odjavljivanje

igrača, pokretanje turnira, formiranje mečeva, unos rezultata, prikaz plasmana i administratorske funkcionalnosti. Pored uspešnih scenarija, proverene su i situacije u kojima sistem treba da odbije nedozvoljenu operaciju. Na osnovu sprovedenih provera potvrđeno je očekivano ponašanje implementiranih funkcionalnosti u testiranim scenarijima.

Rad na projektu predstavljao je praktičnu primenu znanja iz oblasti razvoja web aplikacija, objektno-orijentisanog programiranja, rada sa bazama podataka, autentifikacije i autorizacije, projektovanja poslovne logike i organizacije softverskog sistema. Posebno značajan deo rada predstavlja izdvajanje poslovne logike u servisni sloj, jer je kroz taj proces praktično primenjen princip razdvajanja odgovornosti između različitih komponenti aplikacije.

Dalji razvoj aplikacije mogao bi obuhvatiti naprednije algoritme za generisanje parova, podršku za različite sisteme takmičenja, detaljniju statistiku igrača, istoriju odigranih partija, unapređeni prikaz šahovskih partija i dodatne mogućnosti za organizatore turnira. Takođe bi se vremenska kontrola, koja je u trenutnoj verziji sistema prikazana kao deo interfejsa meča, mogla proširiti stvarnim odbrojavanjem vremena i automatskom obradom isteka vremena.

Na osnovu realizovanih funkcionalnosti može se zaključiti da razvijeni sistem predstavlja funkcionalno rešenje za organizaciju i upravljanje šahovskim turnirima. Primenjena arhitektura, servisni sloj i organizacija baze podataka pružaju odgovarajuću osnovu za održavanje sistema i njegovo dalje proširivanje novim funkcionalnostima.

8. Literatura

- [1] ASP.NET Core MVC – Microsoft Learn <https://learn.microsoft.com/aspnet/core/mvc> [posećeno 15.06.2026.]
- [2] Entity Framework Core – Microsoft Learn <https://learn.microsoft.com/ef/core/> [posećeno 15.06.2026.]
- [3] Bootstrap (UI framework) – W3Schools <https://www.w3schools.com/bootstrap5/> [posećeno 25.06.2026.]
- [4] UML Communication (Collaboration) Diagram – Visual Paradigm Guide <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-iscommunication-diagram/> [posećeno 30.06.2026.]
- [5] FreeCodeCamp – Learn Web Development (MVC osnove kroz praksu) <https://www.freecodecamp.org/news/tag/web-development/> [posećeno 17.06.2026.]
- [6] ASP.NET Core Identity – Microsoft Learn <https://learn.microsoft.com/aspnet/core/security/authentication/identity> [posećeno 03.09.2026.]
- [7] SQL Server – Microsoft Learn <https://learn.microsoft.com/sql/sql-server/> [posećeno 05.09.2026.]
- [8] C# Documentation – Microsoft Learn <https://learn.microsoft.com/dotnet/csharp/> [posećeno 07.09.2026.]
- [9] UML Sequence Diagram – Visual Paradigm Guide <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/> [posećeno 09.09.2026.]
- [10] Entity Framework Core – Microsoft Learn (Relationships) <https://learn.microsoft.com/ef/core/modeling/relationships> [posećeno 12.09.2026.]